SĂPTĂMÂNA 2

Aleatorul în simulare și metoda Monte Carlo

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Unde am rămas

- ✓ Un **model stocastic** conține variabile aleatoare: aceleași intrări → rezultate diferite
- ✓ O singură rulare **nu dovedește nimic**; avem nevoie de multe rulări și de statistică
- ✓ Exemplul bike share: fiecare minut = două „aruncări de monedă” cu probabilitățile p_1 și p_2
- ✓ Azi: **de unde vine** aleatorul și **cum îl folosim** corect

La finalul săptămânii veți putea…

1. să alegeți o **distribuție** potrivită pentru o mărime aleatoare (sosiri, durate, defecte)
2. să calculați **media** și **dispersia** unei distribuții simple
3. să explicați cum funcționează un **generator pseudo-aleator** și rolul **seed-ului**
4. să generați variabile aleatoare prin **transformarea inversă**
5. să estimați o mărime prin **Monte Carlo** și să raportați un **interval de încredere**
6. să estimați **câte simulări** sunt necesare pentru o precizie dată

Ora 1

Probabilități strict necesare

1. Experiment aleator, probabilitate, variabilă aleatoare
2. Distribuție, medie, dispersie
3. Distribuțiile de lucru ale simulării
4. Populație, eșantion, histogramă

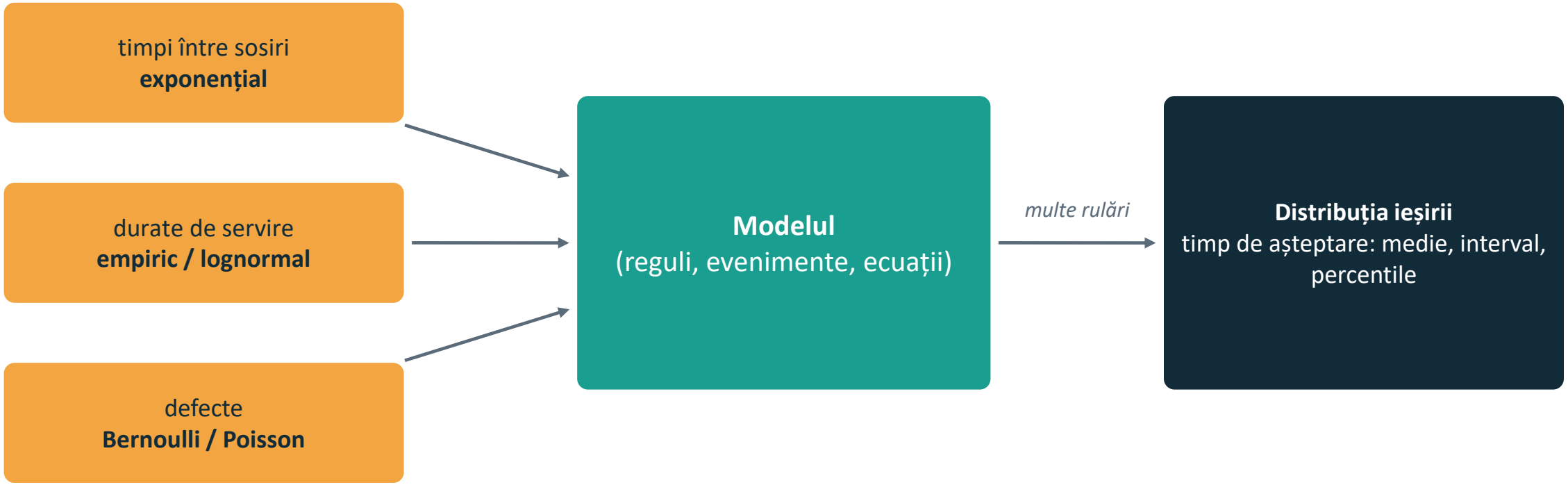
De ce avem nevoie de probabilități?

- Intrările unui sistem real sunt **incerte**: când sosește următorul client? cât durează o cerere?
- Nu putem prezice o valoare anume, dar putem descrie **cât de probabile** sunt valorile
- Descrierea aceasta se numește **distribuție de probabilitate**
- Simularea transformă **distribuțiile intrărilor** în **distribuții ale ieșirilor**

Ideea cheie

Nu știm ce se va întâmpla **data viitoare**, dar știm destul de bine ce se întâmplă **în medie** și **cât de mult variază**.

Simularea stocastică, pe scurt



După principiul simulării Monte Carlo (Holban, cap. 9)

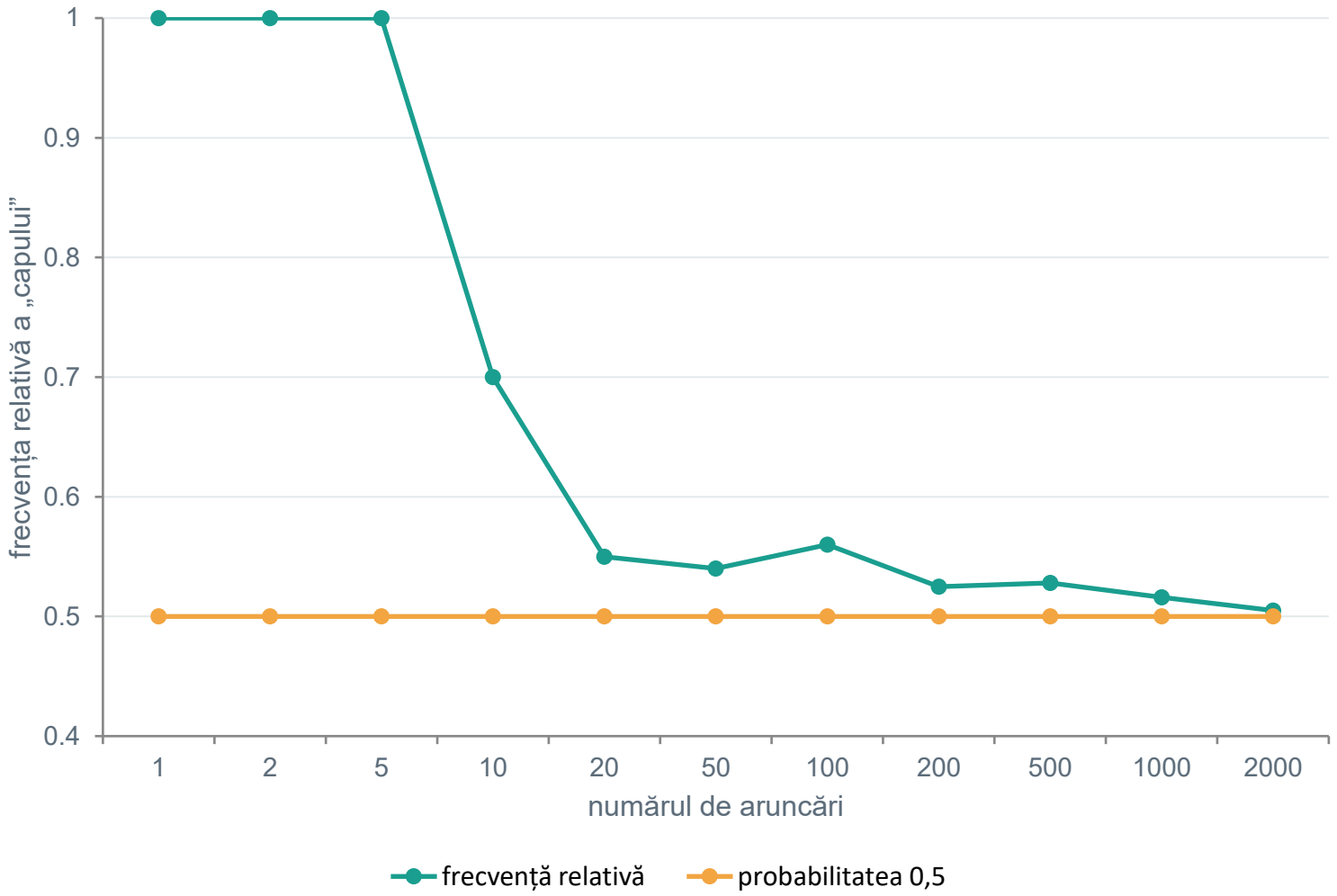
Experiment aleator și probabilitate

Probabilitatea unui eveniment

Un număr între 0 și 1 care măsoară cât de des apare evenimentul dacă repetăm **experimentul aleator** de foarte multe ori.

- Experiment aleator: rezultatul nu poate fi prezis exact (aruncarea unei monede, sosirea unui client)
- Interpretarea frecvențială: P(A) ≈ **frecvența relativă** = de câte ori apare A / numărul de repetări
- Exact ideea simulării: repetăm experimentul în calculator și numărăm

Frecvența relativă se stabilizează



- La început: fluctuații mari (după 10 aruncări: 0,7)
- După 2 000 de aruncări: 0,505
- **Legea numerelor mari**: frecvența relativă → probabilitatea
- Dar convergența este **lentă** – vom cuantifica asta în ora 3

Variabila aleatoare

Variabilă aleatoare

O mărime numerică al cărei rezultat depinde de un experiment aleator – formal, o funcție care asociază un **număr** fiecărui rezultat posibil.

- Numărul de clienți care sosesc într-un minut
- Durata servirii unui client, timpul de răspuns al unui server
- Numărul de calculatoare vândute într-o săptămână (Holban, cap. 9)

Discretă vs continuă

Variabilă discretă

- ia valori **numărabile**: 0, 1, 2, …
- descrisă prin **P(X = x)** pentru fiecare valoare
- ex.: clienți pe minut, pachete pierdute, zaruri
- ex.: calculatoare vândute pe săptămână

vs

Variabilă continuă

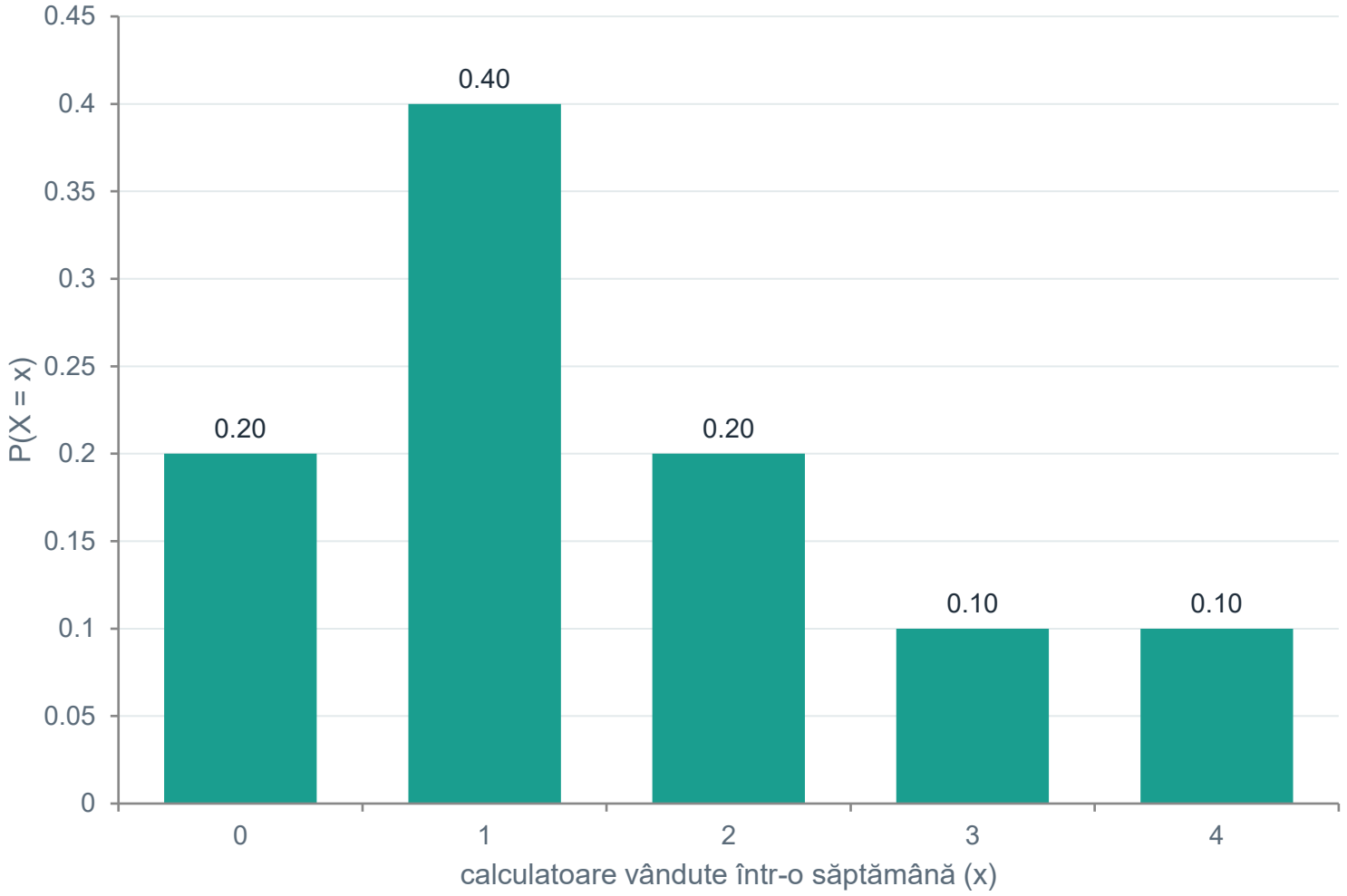
- ia valori **reale** într-un interval
- descrisă printr-o **densitate** f(x)
- ex.: durata servirii, timpul între sosiri
- ex.: temperatura, latența unei cereri

O distribuție discretă din date reale

Calculatoare vândute / săpt.	Nr. de săptămâni	P(X = x)	F(x) = P(X ≤ x)
0	6	0,20	0,20
1	12	0,40	0,60
2	6	0,20	0,80
3	3	0,10	0,90
4	3	0,10	1,00
Total	30	1,00	–

Vânzările unei firme în 30 de săptămâni (Holban, cap. 9, tabelele 9.1–9.2)

Funcția de masă de probabilitate



- **p(x) = P(X = x)** pentru fiecare valoare
- Toate probabilitățile sunt între 0 și 1
- Suma lor este **exact 1**
- Graficul arată imediat valoarea cea mai probabilă (1)

Funcția de repartiție (CDF)

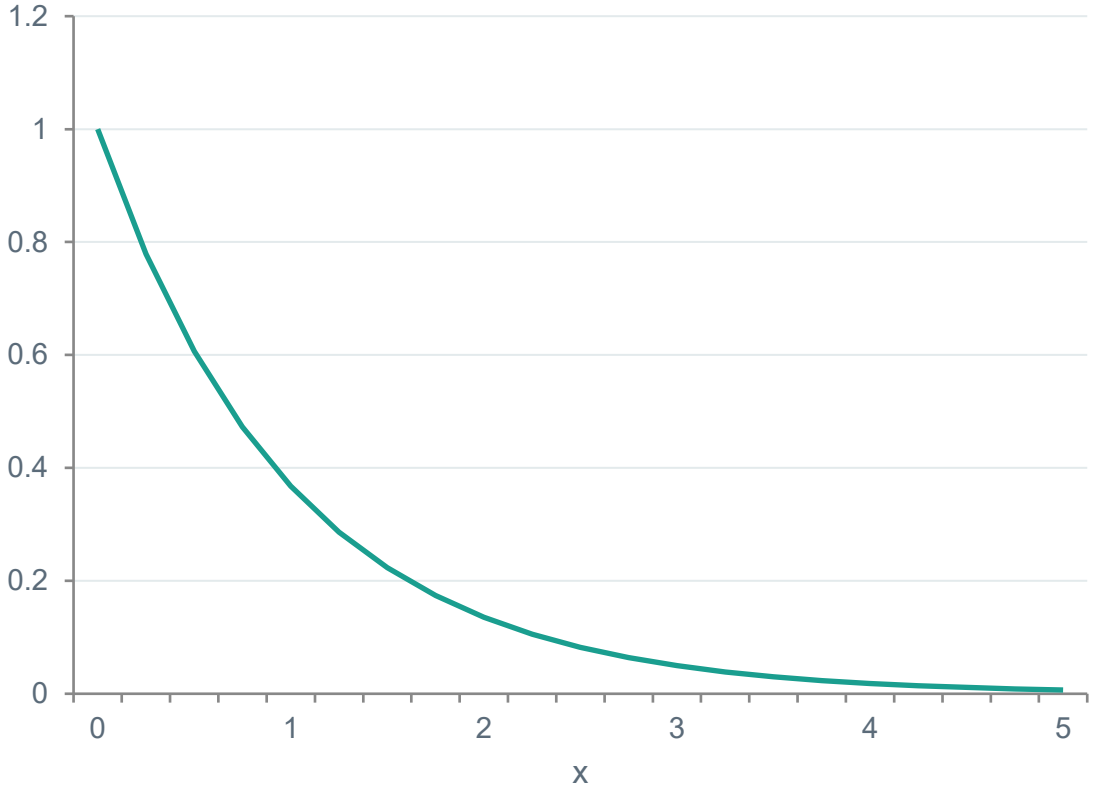
$$F(x) = P(X \leq x)$$

$F(x)$	probabilitatea ca variabila să ia o valoare **cel mult** x
$0 \rightarrow 1$	F crește de la 0 la 1; pentru o variabilă discretă, crește în **trepte**
$P(a<X\leq b)$	= F(b) − F(a): probabilitatea unui interval

Exemplu (vânzări): P(X ≤ 2) = 0,2 + 0,4 + 0,2 = **0,8**. CDF-ul este instrumentul principal pentru **generarea** variabilelor aleatoare (ora 2).

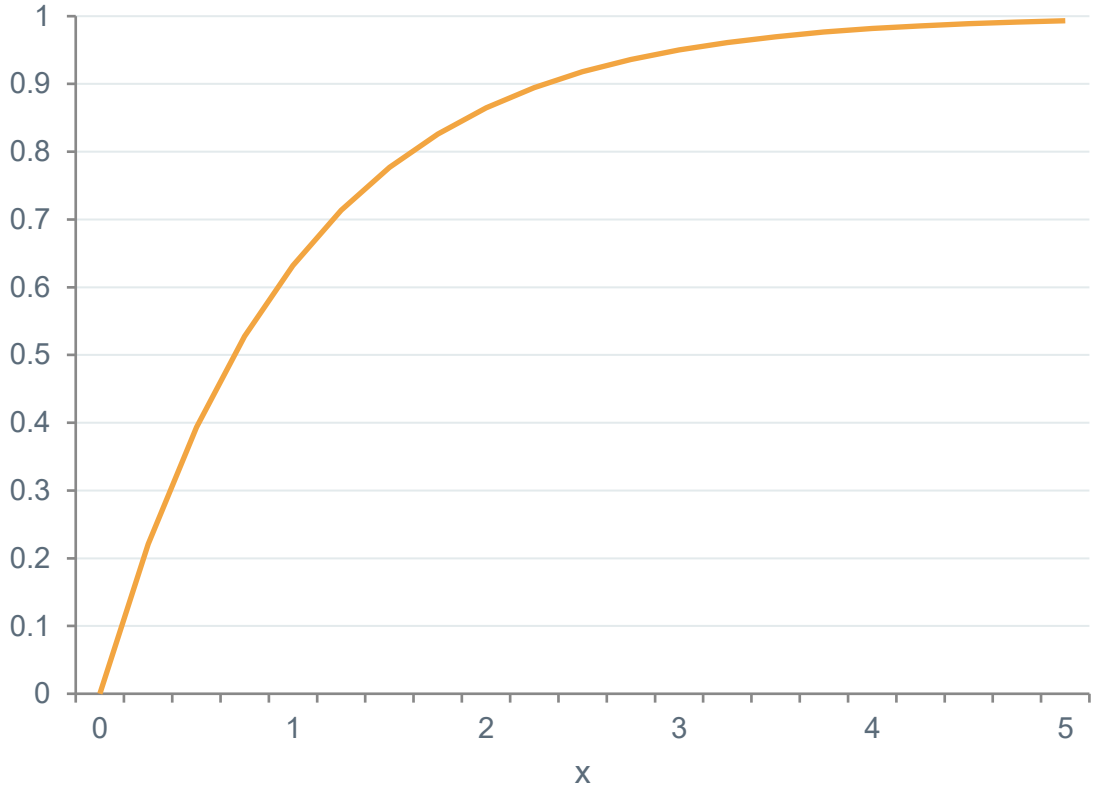
Variabile continue: densitate și repartiție

Densitatea f(x) – exponențiala, λ = 1



probabilitatea unui interval = **aria** de sub curbă

Funcția de repartiție F(x)



F(x) = aria de la stânga lui x

Atenție: densitatea nu este o probabilitate

- Pentru o variabilă continuă, **P(X = x) = 0** pentru orice valoare x exactă
- Are sens doar probabilitatea unui **interval**: P(a ≤ X ≤ b) = aria de sub f între a și b
- f(x) poate fi **mai mare decât 1** (de ex. uniforma pe [0; 0,5] are f = 2)
- Aria totală de sub densitate este întotdeauna **1**

? Intuiție

Care este probabilitatea ca o cerere să dureze **exact** 23,000000… ms? Zero. Dar între 20 și 25 ms? Poate 0,3.

Media (valoarea așteptată)

$$E[X] = \Sigma\, x \cdot P(X = x)$$

discret	suma valorilor ponderate cu probabilitățile lor
continuu	E[X] = ∫ x f(x) dx – aceeași idee, cu integrală
notație	μ (miu) sau E[X]; se mai numește speranță matematică

Vânzări: E[X] = 0·0,2 + 1·0,4 + 2·0,2 + 3·0,1 + 4·0,1 = **1,5 calculatoare pe săptămână**.

Dispersia și abaterea standard

$$Var(X) = E[(X - \mu)^2] = E[X^2] - \mu^2$$

$Var(X)$	cât de mult se **împrăștie** valorile în jurul mediei (σ^2)
σ	abaterea standard = $\sqrt{Var(X)}$, în aceeași unitate ca X
$CV = \sigma/\mu$	coeficientul de variație: variabilitatea relativă, fără unitate

Vânzări: $E[X^2] = 3{,}7$, $Var(X) = 3{,}7 - 1{,}5^2 =$ **1,45**, $\sigma \approx$ **1,204** calculatoare.

De ce contează variabilitatea?

aceeași medie

două servere cu timp mediu de răspuns de 100 ms

CV ≈ 0

primul răspunde **mereu** în ~100 ms

CV ≈ 2

al doilea: des 20 ms, uneori 1 000 ms – utilizatorii se plâng

- În cozi, **variabilitatea** creează așteptare, chiar dacă media e bună (săptămâna 4)
- De aceea raportăm și **percentile** (p95, p99), nu doar media

Verificare

? O variabilă ia valorile 1, 2, 3 cu probabilitățile 0,5, 0,3 și 0,2. Cât este **media** ei?

A 2

B 1,7

C 1,5

D 0,33

Distribuțiile de lucru ale simulării

1 Uniformă

Toate valorile dintr-un interval sunt la fel de probabile. Baza tuturor generatoarelor.

2 Bernoulli / binomială

Succes sau eșec; numărul de succese din n încercări.

3 Poisson

Numărul de evenimente într-un interval: sosiri pe minut, cereri pe secundă.

4 Exponențială

Timpul până la următorul eveniment: între sosiri, până la defectare.

5 Normală

Sume de multe efecte mici: erori de măsură, medii ale rezultatelor.

6 Empirică

Construită direct din date, când nicio formulă nu se potrivește.

Distribuția uniformă U(a, b)

$$f(x) = 1 / (b - a), \quad a \le x \le b$$

media	(a + b) / 2
dispersia	$(b - a)^2 / 12$
U(0, 1)	cazul standard: ce produce un generator de numere aleatoare

Folosim uniforma când știm doar **limitele** unei mărimi (ex. o durată între 4 și 8 zile) și ca **materie primă** pentru toate celelalte distribuții.

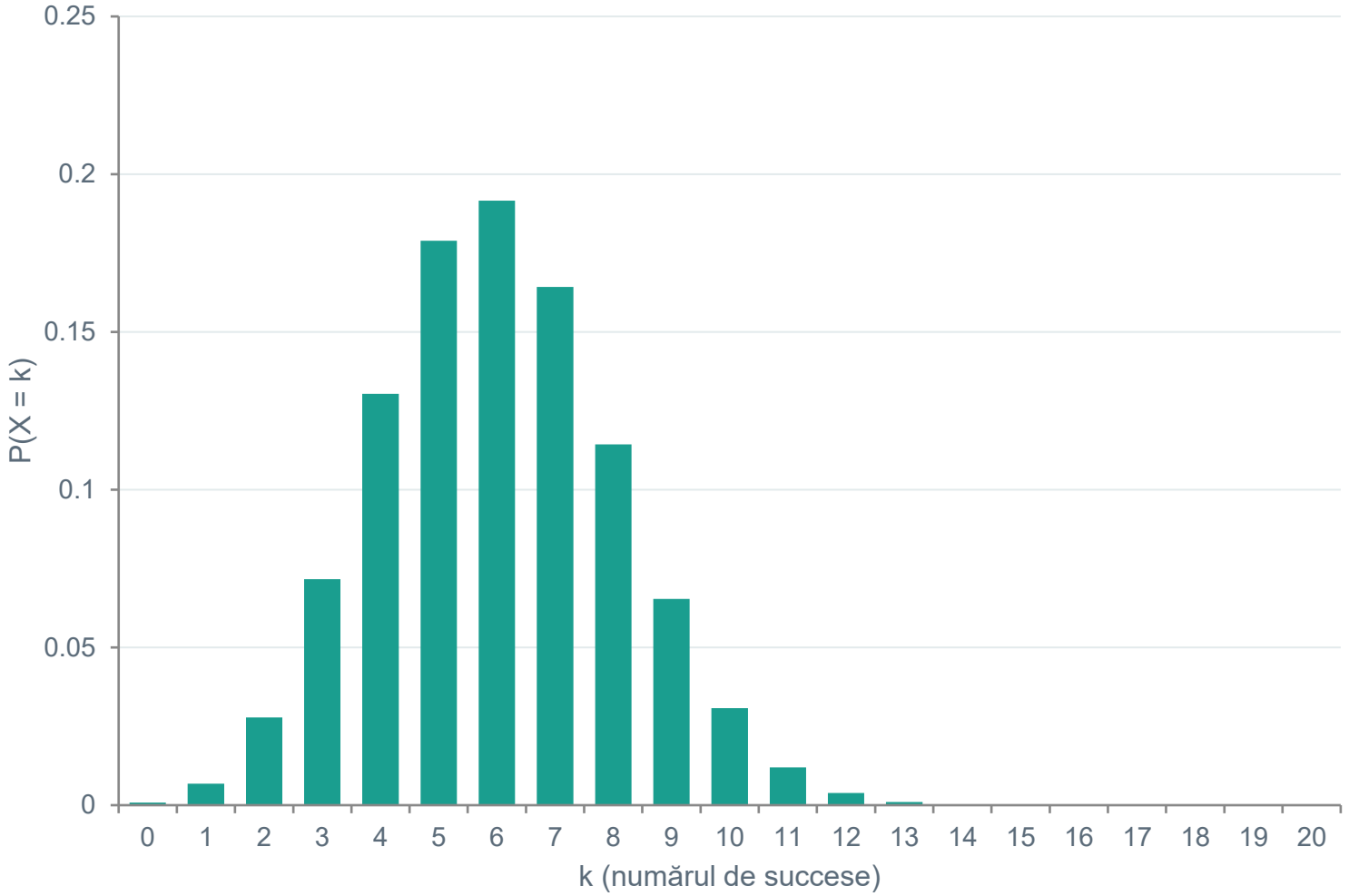
Bernoulli și binomiala

$$P(X = k) = C(n, k) \cdot p^k \cdot (1 - p)^{n-k}$$

Bernoulli(p)	o singură încercare: 1 (succes) cu probabilitatea p, altfel 0; media p
Bin(n, p)	numărul de succese din n încercări independente
medie, disp.	n·p și n·p·(1 − p)

Exemplu: 100 de cereri, fiecare eșuează independent cu p = 0,02 → în medie **2 eșecuri**, dispersie 1,96.

Binomiala Bin(20; 0,3)



- Media n·p = **6**
- Cele mai probabile valori: 5 și 6
- Valori peste 12 sunt practic imposibile
- Forma se apropie de un „clopot” când n crește

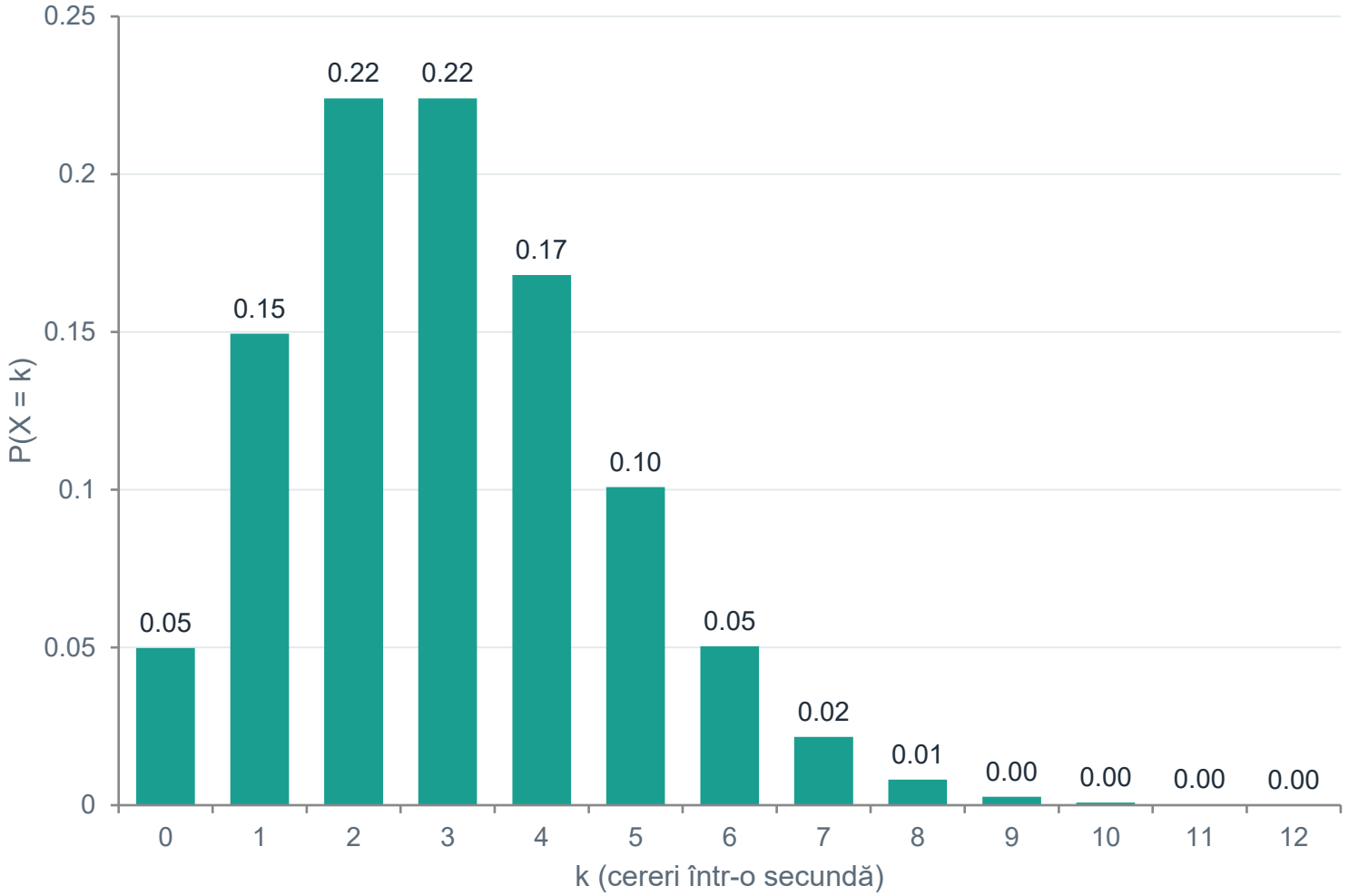
Distribuția Poisson

$$P(X = k) = e^{(-\lambda)} \cdot \lambda^k / k!$$

λ	numărul **mediu** de evenimente în intervalul considerat
medie = disp.	ambele sunt egale cu λ
aproximare	Bin(n, p) ≈ Poisson(n·p) când n e mare și p mic

Folosim Poisson pentru **numărul** de evenimente rare și independente dintr-un interval: cereri pe secundă, apeluri pe minut, defecte pe lună.

Poisson cu λ = 3 cereri pe secundă



- Cel mai des: 2 sau 3 cereri
- $P(0 \text{ cereri}) = e^{-3} \approx 0{,}05$
- $P(X \geq 7) \approx 0{,}03$ – rar, dar se întâmplă
- Dimensionăm serverul pentru **vârfuri**, nu pentru medie

Distribuția exponențială Exp(λ)

$$f(x) = \lambda\, e^{(-\lambda x)}, \quad F(x) = 1 - e^{(-\lambda x)}$$

λ	rata evenimentelor (de ex. 2 sosiri pe minut)
$media$	1/λ (de ex. 0,5 minute între sosiri)
$\sigma = media$	deci coeficientul de variație este exact **1**

Folosim exponențiala pentru **timpul până la următorul eveniment**: timpul dintre sosiri, durata până la o defectare, durata unei convorbiri.

Proprietatea de lipsă a memoriei

- **P(X > t + s | X > t) = P(X > s)**
- Dacă ai așteptat deja 5 minute un client, timpul rămas are **aceeași** distribuție ca la început
- Exponențiala este **singura** distribuție continuă cu această proprietate
- Bună pentru evenimente „fără uzură”: sosiri independente, defecte electronice aleatoare
- Nepotrivită pentru durate cu **uzură** sau cu o durată minimă (o operație chirurgicală)

M **Legătura cu S3**

Lipsa memoriei este exact **proprietatea Markov** pe care o studiem săptămâna viitoare.

Poisson și exponențiala: două fețe ale aceluiași proces

Procesul Poisson cu rata λ

- evenimente independente, rată constantă λ
- **numărul** de evenimente în t: Poisson(λt)
- **timpii dintre** evenimente: Exp(λ)
- model standard pentru sosiri

Exemplu: λ = 2 sosiri pe minut

- într-o oră: în medie **120** de sosiri
- între două sosiri: în medie **30 de secunde**
- P(nicio sosire într-un minut) = $e^{-2} \approx$ **0,135**
- simulare: adunăm timpi exponențiali (ora 2)

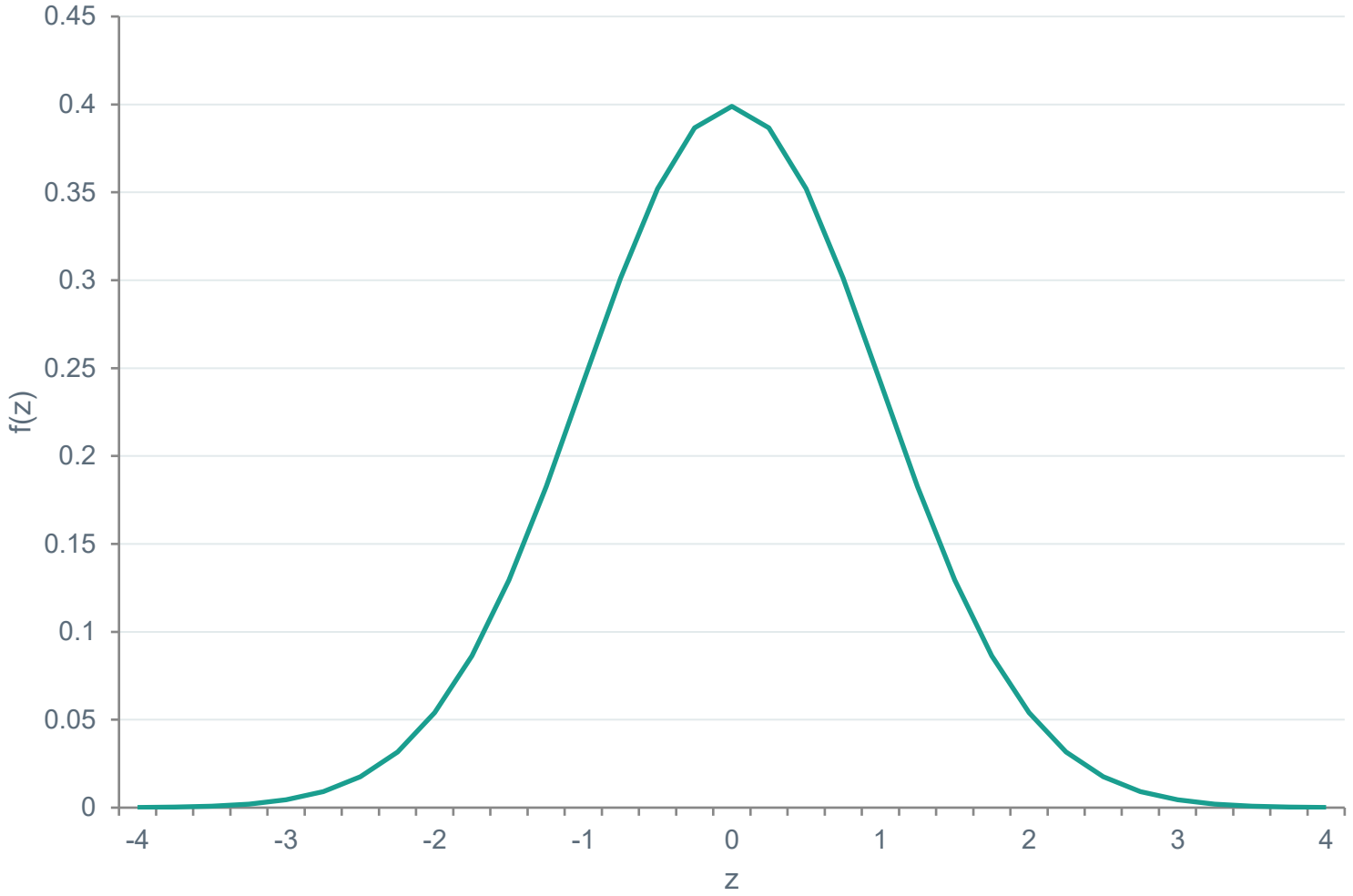
Distribuția normală $N(\mu, \sigma^2)$

$$f(x) = e^{-(x-\mu)^2/2\sigma^2} / (\sigma\sqrt{(2\pi)})$$

μ, σ	media și abaterea standard; curba este simetrică în jurul lui μ
$68–95–99{,}7$	% din valori sunt la cel mult 1, 2, respectiv 3 σ de medie
$Z = (X-\mu)/\sigma$	standardizarea: Z are distribuția N(0, 1)

Apare când o mărime este **suma multor efecte mici** independente – și, foarte important, pentru **mediile** rezultatelor unei simulări (ora 3).

Normala standard N(0, 1)



- Între −1 și 1: **68%** din valori
- Între −1,96 și 1,96: **95%**
- Între −3 și 3: **99,7%**
- Valoarea 1,96 o vom folosi la intervalele de încredere

Distribuții cu „coadă grea”

- Multe mărimi din informatică au **valori extreme mult mai frecvente** decât prevede normala
- Exemple: dimensiunea fișierelor, durata sarcinilor, latența rețelei, popularitatea paginilor
- Modele tipice: **lognormală**, **Pareto**, Weibull
- Consecință: media poate fi dominată de câteva valori uriașe; percentilele devin esențiale

Atenție

Dacă înlocuiești o distribuție cu coadă grea cu o exponențială „pentru simplitate”, simularea va **subestima** cozile și întârzierile.

Distribuția empirică

Distribuție empirică

Distribuția construită **direct din date**: fiecare valoare observată (sau fiecare interval) primește probabilitatea egală cu **frecvența ei relativă**.

- Exemplul vânzărilor de calculatoare este o distribuție empirică
- Avantaj: nu presupune nicio formulă; reproduce exact datele
- Dezavantaj: nu generează valori pe care nu le-am observat niciodată (ex. vârfuri extreme)

Cum aleg distribuția? Primă orientare

Mărimea modelată	Distribuție	Parametri
Știu doar un minim și un maxim	uniformă	a, b
Știu minim, cel mai probabil, maxim	triunghiulară	a, m, b
Succes / eșec al unei încercări	Bernoulli	p
Număr de evenimente într-un interval	Poisson	λ
Timp între evenimente independente	exponențială	λ (sau media)
Sumă / medie a multor efecte	normală	μ, σ
Am multe date, nicio formulă nu se potrivește	empirică	datele

Exercițiu: ce distribuție ați alege?

Mărimea	Distribuția?
Numărul de e-mailuri primite de un server într-un minut	?
Timpul până la defectarea unui disc (fără uzură)	?
Dacă o tranzacție eșuează sau nu	?
Durata unei activități: minim 4, cel mai probabil 6, maxim 8 zile	?
Eroarea medie de măsurare, obținută din 50 de măsurători	?

Populație și eșantion

Populația

- **toate** valorile posibile ale fenomenului
- descrisă de parametri: **μ**, **σ**
- de obicei necunoscută
- ex.: toate cererile pe care le va primi serverul

Eșantionul

- valorile pe care le **observăm** efectiv
- descris de statistici: **$\bar{x}$**, **s**
- estimează parametrii populației
- trebuie să fie **reprezentativ** (ales aleator)

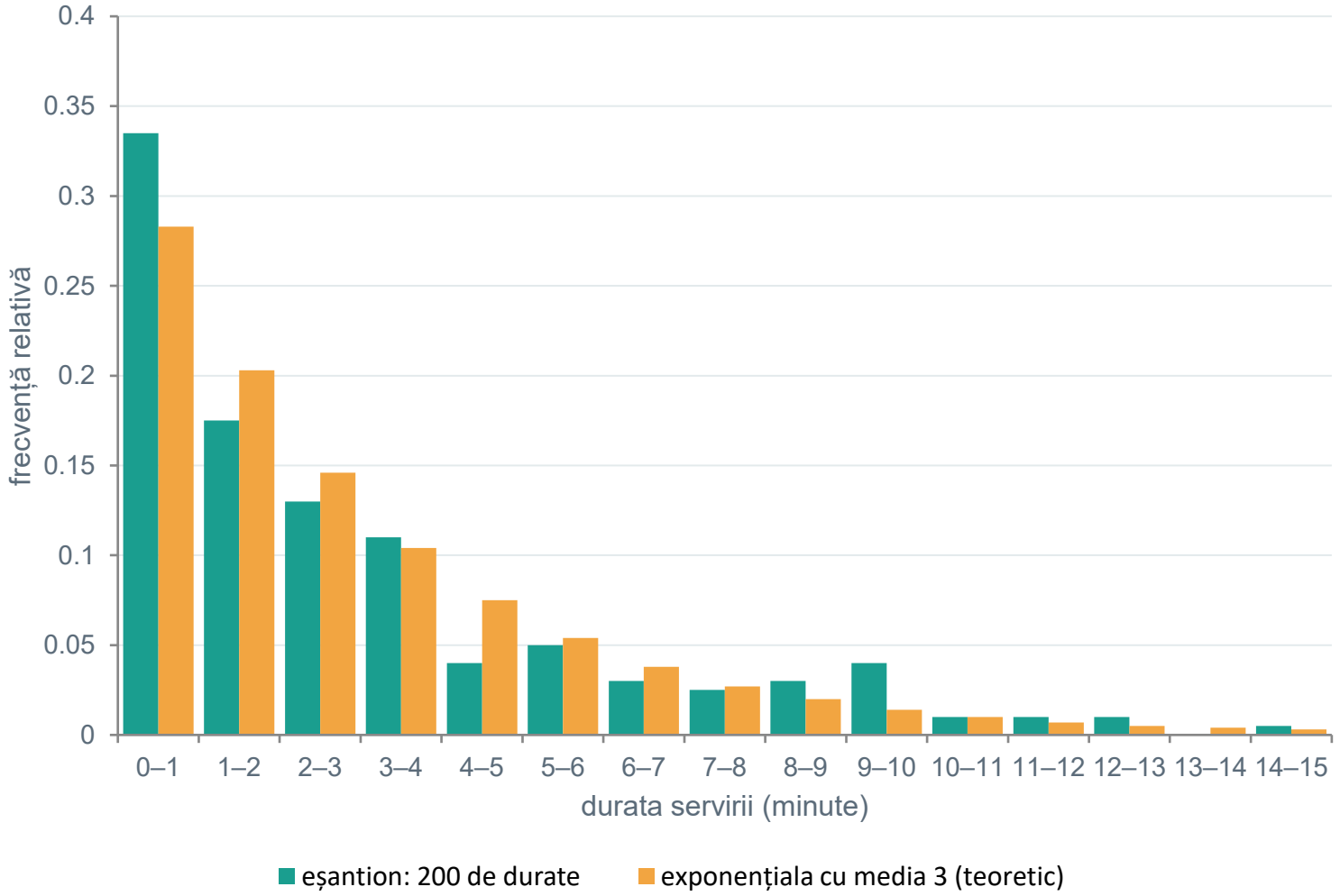
Media și dispersia de selecție

$$\bar{x} = (1/n)\,\Sigma\, x_i \qquad s^2 = \Sigma\,(x_i - \bar{x})^2 / (n - 1)$$

$\bar{x}$	media de selecție – estimează μ
s^2	dispersia de selecție – estimează σ²
$n - 1$	împărțim la n − 1, nu la n: corecția Bessel (estimare nedeplasată)

În NumPy: **np.mean(x)** și **np.std(x, ddof=1)** – fără ddof=1 se împarte la n.

Histograma: distribuția văzută din date



- Media de selecție: **3,02** min (teoretic 3)
- Abaterea standard: **3,05** min (teoretic 3)
- Histograma seamănă cu densitatea, dar are „zgomot”
- Cu mai multe date, zgomotul scade

Verificare

? Ce distribuție folosim de obicei pentru **timpul dintre două sosiri** consecutive ale clienților, când sosirile sunt independente și au o rată constantă?

A Normală

B Uniformă

C Exponențială

D Binomială

Verificare

? Numărul de cereri pe secundă primite de un server este Poisson cu $\lambda = 4$. Cât sunt **media** și **dispersia**?

A media 4, dispersia 2

B media 4, dispersia 16

C media 2, dispersia 4

D media 4, dispersia 4

Verificare

? Durata unei cereri este o variabilă **continuă**. Cât este probabilitatea ca durata să fie **exact** 20 ms?

A f(20)

B 0

C F(20)

D 1 – F(20)

Verificare

? O monedă corectă a căzut de **5 ori la rând** „cap”. Care este probabilitatea ca la a 6-a aruncare să cadă tot „cap”?

A mai mică de 0,5 – „trebuie” să vină pajura

B 0,5

C mai mare de 0,5 – moneda e „pe val”

D 1/64

Rezumatul orei 1

- ✓ Probabilitatea ≈ **frecvența relativă** pe termen lung (legea numerelor mari)
- ✓ Variabile **discrete** (P(X = x)) și **continue** (densitate f, probabilitate = arie)
- ✓ **CDF** F(x) = P(X ≤ x) – instrumentul pentru generare
- ✓ **Media** și **dispersia**: nivelul și împrăștierea; variabilitatea contează
- ✓ Distribuții de lucru: uniformă, Bernoulli/binomială, **Poisson**, **exponențială**, normală, empirică
- ✓ Din date: eșantion, **x̄**, **s**, histogramă

Ora 2

De unde vine aleatorul în calculator

1. Aleator adevărat vs pseudo-aleator
2. Generatoare congruențiale, Mersenne Twister, PCG64
3. Seed, reproductibilitate, fluxuri paralele
4. De la U(0, 1) la orice distribuție

Unde am rămas

- ✓ Intrările aleatoare ale unui model sunt descrise prin **distribuții**
- ✓ **CDF** $F(x) = P(X \leq x)$ caracterizează complet o distribuție
- ✓ Acum: cum obținem în calculator **valori concrete** din aceste distribuții?





Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin.

— John von Neumann, 1951

Un algoritm determinist nu poate produce aleator adevărat. Dar poate produce secvențe care **trec toate testele statistice** – și asta ne ajunge.

Aleator adevărat vs pseudo-aleator

Aleator adevărat (TRNG)

- sursă **fizică**: zgomot termic, radioactivitate, jitter
- în calculator: /dev/random, instrucțiunea RDRAND
- **imprevizibil**, dar lent și nereproductibil
- folosit în **criptografie** (chei, parole)

VS

Pseudo-aleator (PRNG)

- **algoritm determinist** pornit dintr-un seed
- rapid: sute de milioane de numere pe secundă
- **reproductibil**: același seed → aceeași secvență
- folosit în **simulare**, jocuri, ML

Ce cerem de la un generator pentru simulare

1 Uniformitate

Valorile acoperă egal intervalul [0, 1).

2 Independență

O valoare nu „trădează” următoarea; fără tipare.

3 Perioadă lungă

Secvența se repetă abia după foarte multe numere.

4 Viteză

O simulare poate cere miliarde de numere.

5 Reproductibilitate

Același seed dă exact aceeași secvență – esențial la depanare.

6 Fluxuri independente

Pentru rulări paralele, secvențe care nu se suprapun.

Generatorul congruențial liniar (LCG)

$$X_{n+1} = (a \cdot X_n + c) \bmod m, \qquad U_n = X_n / m$$

X_0	seed-ul (valoarea de pornire)
a, c, m	multiplicator, increment, modul – aleși cu grijă
U_n	numărul „uniform” din [0, 1)

Perioada este cel mult **m**. Cu c = 0 obținem generatorul multiplicativ; clasicul „minimal standard”: m = $2^{31} - 1$, a = 16 807.

Un LCG de jucărie: a = 5, c = 3, m = 16, $X_0 = 7$

n	X_n	$U_n = X_n/16$	Observație
0	7	0,4375	seed
1	6	0,3750	(5·7 + 3) mod 16 = 38 mod 16 = 6
2	1	0,0625	
3	8	0,5000	
4	11	0,6875	
5	10	0,6250	
…	…	…	
16	7	0,4375	revine la 7: perioada = **16** (maximă)

LCG în Python

```
def lcg(a, c, m, seed):
    x = seed
    while True:
        x = (a * x + c) % m
        yield x / m           # U în [0, 1)

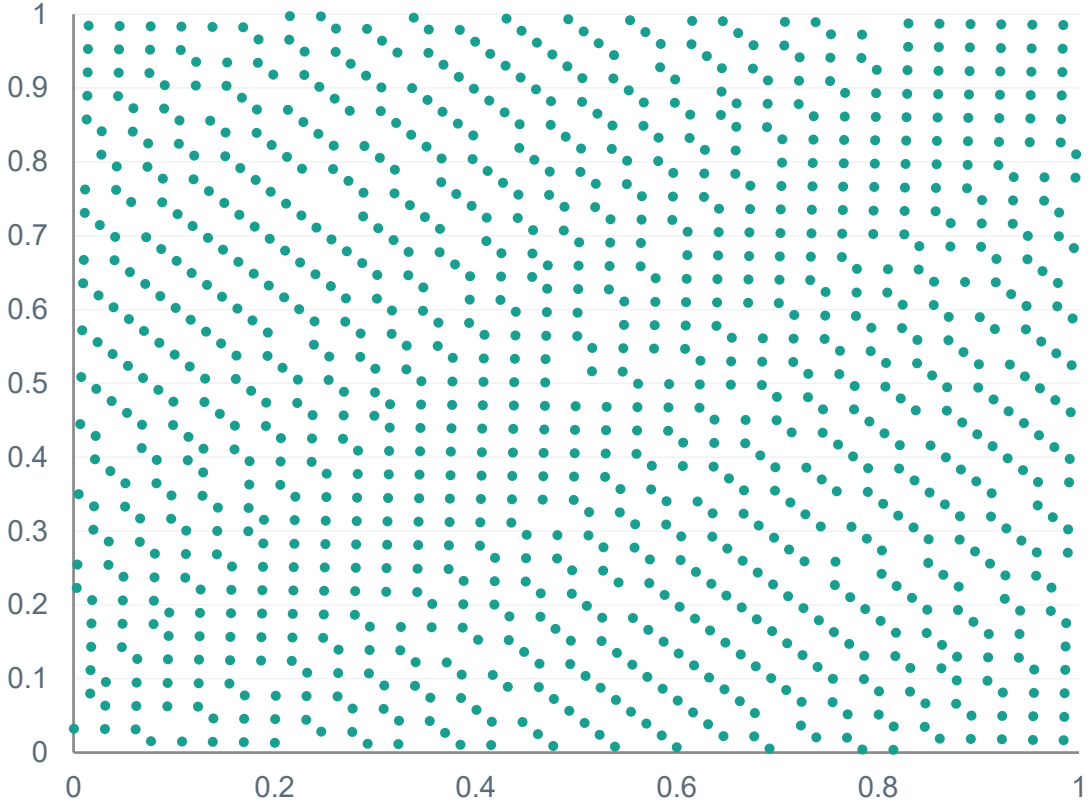
g = lcg(5, 3, 16, seed=7)
print([round(next(g), 4) for _ in range(5)])
# [0.375, 0.0625, 0.5, 0.6875, 0.625]

# o alegere proastă: a = 5, c = 0, m = 16, seed = 1
# 1, 5, 9, 13, 1, 5, 9, 13 ... perioada 4!
```

- Un generator Python cu **yield**
- Aceeași secvență la fiecare pornire cu seed = 7
- Parametrii contează enorm: cu c = 0 perioada scade la **4**
- Nu inventați generatoare – folosiți-le pe cele testate

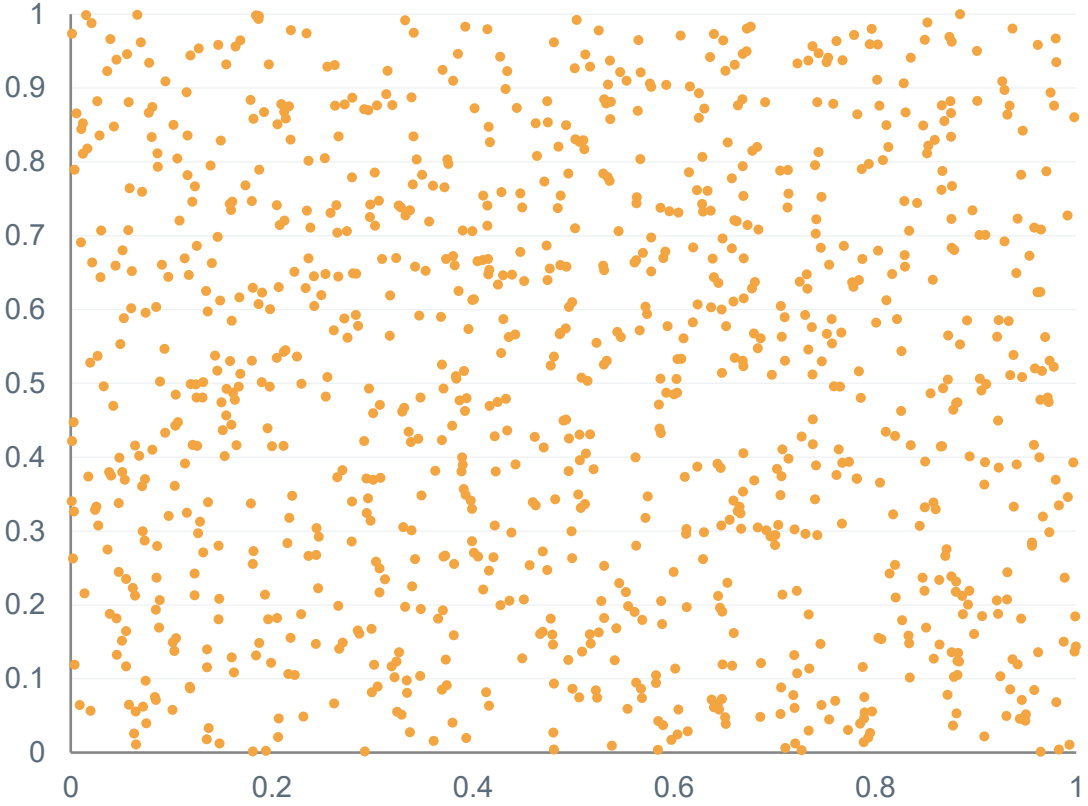
Tipare ascunse: perechi (U_n, U_{n+1})

LCG cu m = 2048 (a = 65, c = 1)



punctele cad pe **drepte paralele**

PCG64 (generatorul implicit din NumPy)



fără structură vizibilă

O poveste celebră: RANDU

- Generator IBM din anii 1960: $\mathbf{X_{n+1} = 65\,539 \cdot X_n \bmod 2^{31}}$
- Folosit ani la rând în simulări științifice
- Tripletele consecutive (U_n, U_{n+1}, U_{n+2}) cad pe doar **15 plane** în cubul unitate
- Rezultatele multor simulări din acea perioadă au fost puse sub semnul întrebării

Lecția

Un generator poate arăta bine în 1D (histogramă uniformă) și totuși să fie **foarte prost** în mai multe dimensiuni.

Generatoare folosite astăzi

Generator	Perioada	Unde îl întâlniți	Observații
LCG (minimal standard)	$\approx 2{,}1 \cdot 10^9$	implementări vechi	epuizat în câteva secunde de un CPU modern
Mersenne Twister (1998)	$2^{19937} - 1$	modulul random din Python, MATLAB	foarte bun statistic; stare mare; previzibil
PCG64 (2014)	2^{128}	NumPy: default_rng()	rapid, stare mică, fluxuri independente
CSPRNG	–	secrets, os.urandom	imprevizibil; pentru securitate, nu pentru simulare

Simulare ≠ securitate

- Un PRNG pentru simulare este **previzibil**: din câteva ieșiri se poate reconstrui starea
- Mersenne Twister: după 624 de valori consecutive, **toate** valorile viitoare sunt cunoscute
- Pentru parole, token-uri, chei: **secrets** (Python) sau **os.urandom**
- Invers: pentru simulare nu folosiți CSPRNG – e mai lent și **nereproductibil**

S **Regula**

random / numpy.random → simulare. **secrets** → securitate. Niciodată invers.

Seed-ul și reproductibilitatea

Seed (sămânța generatorului)

Valoarea inițială a stării generatorului. **Același seed → exact aceeași secvență** de numere, deci exact aceeași simulare.

- Esențial pentru **depanare**: reproducem exact rularea cu problema
- Esențial pentru **reproductibilitate**: altcineva obține aceleași rezultate
- Seed-urile folosite se trec în **raportul** simulării

Generatoare în Python și NumPy

```
import random
import numpy as np

random.seed(42)                    # Mersenne Twister
print(random.random())             # 0.6394...

rng = np.random.default_rng(42) # PCG64 (recomandat)
print(rng.random(3))               # 3 uniforme în [0, 1)
print(rng.integers(1, 7, 10))   # 10 aruncări de zar

# aceeași valoare de seed -> aceleași numere
rng2 = np.random.default_rng(42)
print(rng2.random(3))              # identic cu mai sus
```

- **default_rng(seed)** este modul recomandat în NumPy
- Un obiect generator per simulare, nu variabile globale
- Vectorizare: rng.random(1_000_000) într-un singur apel
- np.random.seed() este API-ul vechi

Bune practici cu seed-uri

- Fixați seed-ul **o singură dată**, la începutul simulării – nu în interiorul buclei
- Nu folosiți ora curentă ca seed dacă vreți reproductibilitate
- Replicări diferite → **fluxuri independente**, nu seed-uri 1, 2, 3 alese la întâmplare
- În NumPy: **SeedSequence(...).spawn(n)** produce n fluxuri independente
- Notați seed-ul principal și versiunea bibliotecilor

Greșeală tipică

random.seed(42) apelat **în fiecare pas** al buclei: toate „aruncările” devin identice.

Fluxuri independente pentru replicări (și în paralel)

```
import numpy as np

ss = np.random.SeedSequence(2026)  # seed principal
fluxuri = [np.random.default_rng(s)
           for s in ss.spawn(30)]      # 30 de fluxuri

def o_replicare(rng):
    sosiri = rng.exponential(0.5, size=1000)
    return sosiri.mean()

rezultate = [o_replicare(r) for r in fluxuri]
# fiecare replicare are propriul flux, independent,
# și poate rula pe alt nucleu de procesor
```

- Un singur seed principal → **30 de fluxuri** statistic independente
- Fiecare replicare primește generatorul ei
- Merge identic în serie sau în paralel
- Rezultatele rămân reproductibile

Greșeala clasică: seed-ul în buclă

```
import random

# GREȘIT: resetăm generatorul la fiecare pas
for i in range(5):
    random.seed(42)
    print(random.random())   # 0.639... de 5 ori!

# CORECT: seed o singură dată
random.seed(42)
for i in range(5):
    print(random.random())   # 5 valori diferite
```

- Cu seed în buclă, fiecare „aruncare” dă **același** număr
- Simularea pare să meargă, dar aleatorul a dispărut
- Același efect: un generator nou creat în fiecare funcție apelată
- Test simplu: afișați primele valori generate

Cum se testează un generator?

1 Uniformitate

Histograma și testul χ^2: fiecare subinterval primește cam aceeași proporție.

2 Independență

Autocorelația, testul run-urilor, graficele de perechi și triplete.

3 Baterii de teste

TestU01 (BigCrush), PractRand: sute de teste statistice automate.

4 Practic

Folosim generatoare care **au trecut** aceste baterii – nu le testăm noi de fiecare dată.

Verificare

? LCG cu a = 5, c = 3, m = 16 și $X_0 = 7$. Cât este $\mathbf{X_1}$?

A 38

B 6

C 7

D 2

De la U(0, 1) la orice distribuție

1 Transformarea inversă

$X = F^{-1}(U)$. Metoda generală, când putem inversa CDF-ul.

2 Acceptare – respingere

Propunem valori și le păstrăm pe unele. Merge pentru orice densitate mărginită.

3 Compunere / sume

Ex.: suma a n exponențiale → Erlang; amestecuri de distribuții.

4 Metode speciale

Ex.: Box–Muller pentru normală; algoritmi dedicați în biblioteci.

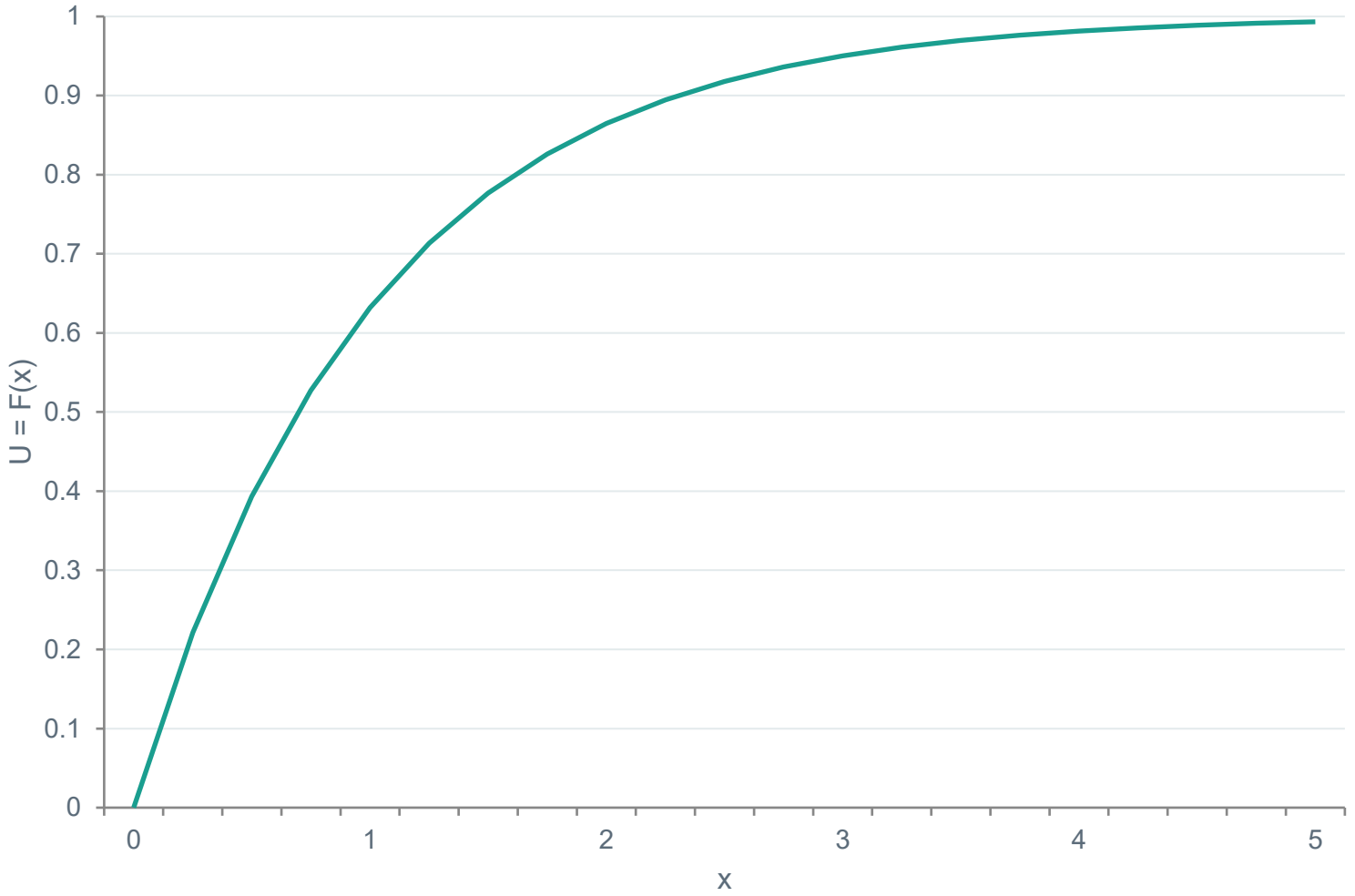
Transformarea inversă

$$X = F^{-1}(U), \quad U \sim U(0, 1)$$

ideea	U este o probabilitate aleatoare; căutăm x pentru care F(x) = U
de ce merge	$P(X \le x) = P(F^{-1}(U) \le x) = P(U \le F(x)) = F(x)$
condiție	trebuie să putem calcula (sau aproxima) inversa lui F

Intuiție: „întindem” uniformele pe axa x după forma CDF-ului – unde F crește repede (densitate mare) ajung multe valori.

Transformarea inversă, vizual



- Alegem U aleator pe axa **verticală**
- Mergem orizontal până la curbă
- Coborâm pe axa **orizontală**: acesta este x
- Ex.: U = 0,5 → x = ln 2 ≈ 0,69 (mediana)

Exemplu: distribuția exponențială

$$U = 1 - e^{(-\lambda X)} \quad \Rightarrow \quad X = -\ln(1 - U) / \lambda$$

simplificare 1 − U este tot uniformă, deci putem folosi X = −ln(U) / λ

exemplu λ = 2 sosiri/min, U = 0,5 → X = −ln(0,5)/2 ≈ **0,3466 minute**

verificare media valorilor generate trebuie să fie ≈ 1/λ = 0,5

Aceasta este exact funcția folosită de bibliotecile de simulare pentru a genera timpii dintre sosiri.

Verificăm transformarea inversă

```
import numpy as np

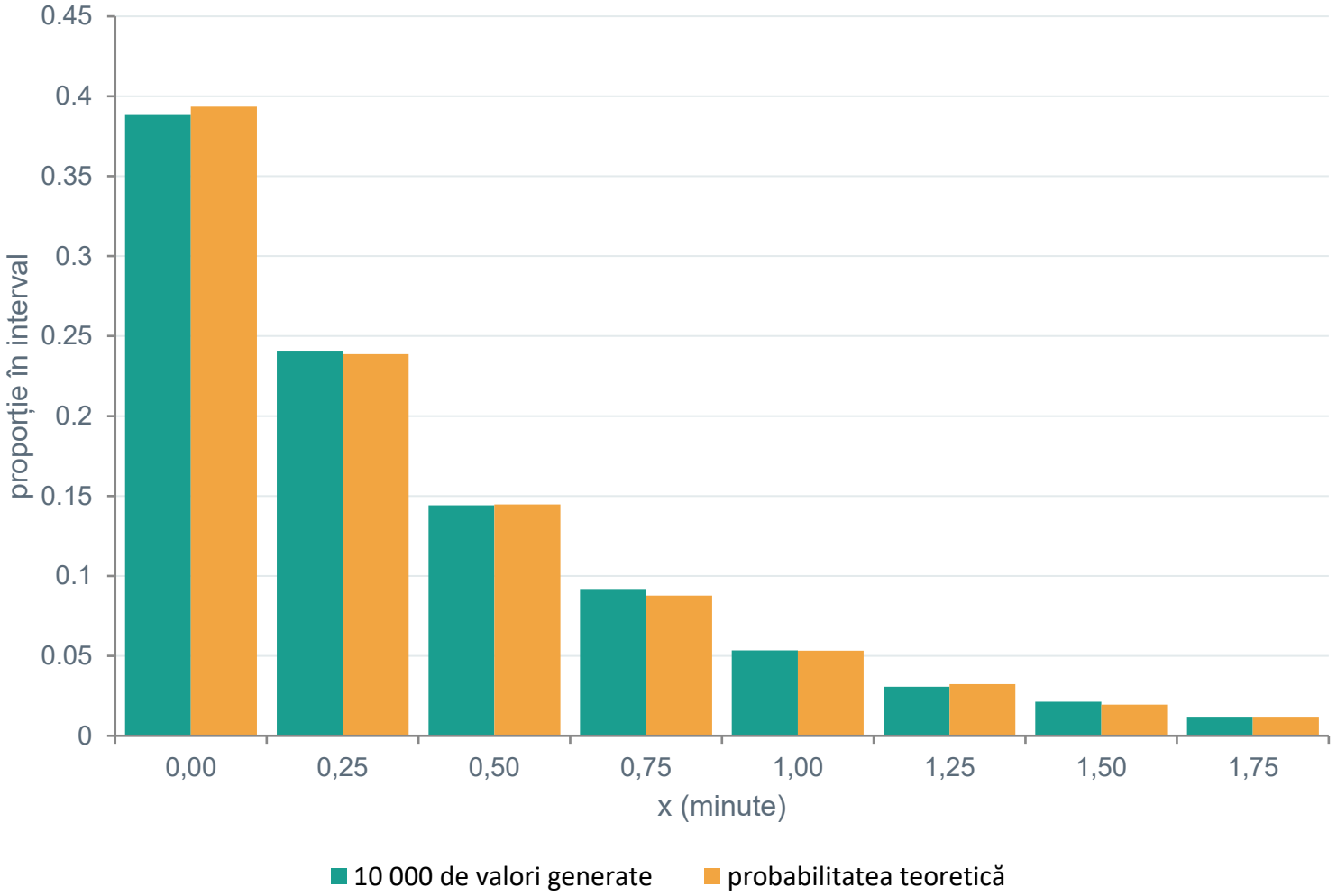
rng = np.random.default_rng(42)
lam = 2.0
U = rng.random(10_000)
X = -np.log(U) / lam      # transformarea inversă

print(X.mean())           # ≈ 0.503 (teoretic 0.5)

# echivalent, direct din bibliotecă:
Y = rng.exponential(scale=1/lam, size=10_000)
```

- Media obținută: **0,503** (teoretic 0,5)
- O linie de cod, fără bucle
- Biblioteca face același lucru intern
- Verificăm **întotdeauna** media și histograma

Histograma valorilor generate vs teoria



- Barele se potrivesc foarte bine
- Diferențele mici sunt **zgomot de eșantionare**
- Același test se face pentru orice generator de variabile

Uniforma pe [a, b] și uniforma discretă

$$X = a + (b - a)\cdot U \qquad K = \lfloor a + (b - a + 1)\cdot U \rfloor$$

continuu o durată între 4 și 8 zile: X = 4 + 4·U

discret un întreg între a și b, toate cu aceeași probabilitate

exemplu vânzări între 6 și 10 pe zi, U = 0,709084 → ⌊6 + 5·0,709⌋ = **9** (Holban)

În NumPy: **rng.uniform(a, b)** și **rng.integers(a, b + 1)** (limita superioară este exclusă).

Transformarea inversă pentru o distribuție discretă

Vânzări x	P(X = x)	F(x)	Intervalul lui U
0	0,20	0,20	[0,0; 0,2)
1	0,40	0,60	[0,2; 0,6)
2	0,20	0,80	[0,6; 0,8)
3	0,10	0,90	[0,8; 0,9)
4	0,10	1,00	[0,9; 1,0)

Regula: X = cea mai mică valoare x pentru care U < F(x) (Holban, tabelul 9.4)

Simulăm 10 săptămâni de vânzări

Săptămâna	U	Interval	Vânzări simulate
1	0,486281	[0,2; 0,6)	1
2	0,928927	[0,9; 1)	4
3	0,205136	[0,2; 0,6)	1
4	0,852845	[0,8; 0,9)	3
5	0,005645	[0; 0,2)	0
6	0,651622	[0,6; 0,8)	2
7	0,799931	[0,6; 0,8)	2
8	0,333693	[0,2; 0,6)	1
9	0,416841	[0,2; 0,6)	1
10	0,011913	[0; 0,2)	0

Media simulată: 1,5 (teoretic 1,5); abaterea standard: 1,269 (Holban, tabelul 9.5)

Verificare

? Folosind tabelul de intervale al vânzărilor (F = 0,2; 0,6; 0,8; 0,9; 1,0), ce vânzare generăm pentru **U = 0,65**?

A 0

B 1

C 2

D 3

Distribuții discrete și empirice în cod

```
import numpy as np
rng = np.random.default_rng(1)

valori = np.array([0, 1, 2, 3, 4])
p      = np.array([.2, .4, .2, .1, .1])

# 1) transformarea inversă „de mână”
F = np.cumsum(p)                    # [.2 .6 .8 .9 1.]
U = rng.random(10)
X = valori[np.searchsorted(F, U, side="right")]

# 2) direct din bibliotecă
Y = rng.choice(valori, size=10, p=p)

# 3) empiric: re-eșantionăm direct datele observate
date = np.array([2.1, 3.4, 1.9, 7.8, 2.5])
Z = rng.choice(date, size=10)
```

- **cumsum** construiește F; **searchsorted** caută intervalul
- **rng.choice** face totul într-un apel
- Distribuție empirică: alegem aleator dintre observații
- Toate variantele sunt vectorizate

Metoda acceptare – respingere

Când F^{-1} nu are formulă: „aruncăm” puncte sub o anvelopă și păstrăm doar pe cele de sub f(x).

1

Alegem o anvelopă

c·g(x) ≥ f(x) peste tot; g ușor de generat (ex. uniformă)

→

2

Propunem

generăm Y din g și U uniform

→

3

Testăm

acceptăm Y dacă U·c·g(Y) ≤ f(Y)

→

4

Repetăm

altfel respingem și încercăm din nou

Exemplu: f(x) = 6x(1 − x) pe [0, 1]

c = 1,5

maximul lui f (în x = 0,5); anvelopa: dreptunghiul de înălțime 1,5

66,7%

din propuneri acceptate (teoretic 1/c = 66,7%)

0,499

media valorilor acceptate (teoretic 0,5)

- Eficiența = **1/c**: cu cât anvelopa e mai „strânsă”, cu atât respingem mai puțin
- Aceeași idee apare în estimarea lui π din ora 3

Distribuția normală: Box–Muller

$$Z_1 = \sqrt{(-2 \ln U_1)} \cdot \cos(2\pi U_2)$$

Z_2	= √(−2 ln U_1) · sin(2π U_2) – o a doua normală, independentă
intrare	două uniforme independente U_1, U_2
$N(\mu, \sigma^2)$	X = μ + σ·Z

CDF-ul normalei nu are inversă în formă închisă; Box–Muller (1958) ocolește problema. Bibliotecile moderne folosesc variante mai rapide (ex. metoda ziggurat).

În practică: generatoarele din NumPy

```
rng = np.random.default_rng(2026)

rng.uniform(4, 8, size=5)           # U(4, 8)
rng.integers(6, 11, size=5)         # întregi 6..10
rng.exponential(scale=0.5, size=5) # media 0.5
rng.poisson(lam=3, size=5)          # cereri/secundă
rng.normal(loc=100, scale=15, size=5)
rng.binomial(n=100, p=0.02, size=5)
rng.triangular(4, 6, 8, size=5)     # min, mod, max
rng.lognormal(mean=0, sigma=1, size=5)
```

- Toate distribuțiile din ora 1, gata implementate
- Atenție la parametri: **scale = 1/λ** la exponențială
- Înțelegem principiul pentru distribuții **proprii** și pentru depanare
- scipy.stats: CDF, PDF, inverse, potriviri

Generarea unui proces Poisson de sosiri

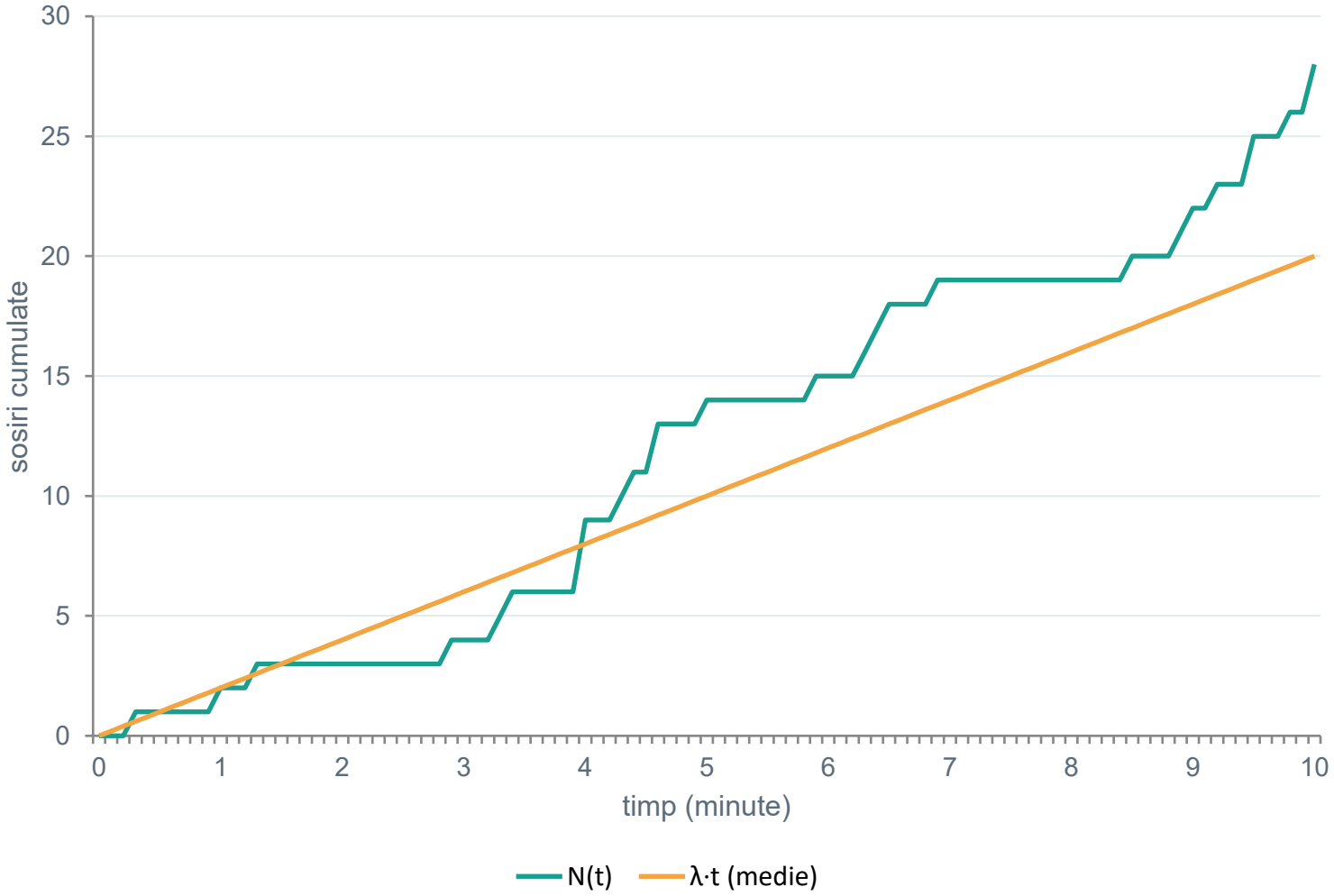
```
rng = np.random.default_rng(8)
lam, T = 2.0, 10.0      # 2 sosiri/min, 10 min

t, sosiri = 0.0, []
while True:
    t += rng.exponential(1 / lam)  # următoarea
    if t > T:
        break
    sosiri.append(t)

print(len(sosiri))          # în medie λ·T = 20
```

- Adunăm timpi exponențiali între sosiri
- Oprim la orizontul T
- În această rulare: **28** sosiri (în medie 20)
- Baza oricărui simulator de cozi (S4–S5)

Numărul de sosiri N(t) într-o rulare



- N(t) crește în **trepte** de câte 1
- A doua linie: media λ·t
- Pauzele lungi și „ciorchinii” sunt normali într-un proces Poisson

Verificare

? Folosind transformarea inversă $X = -\ln(U)/\lambda$, cu $\lambda = 2$ și $U = 0{,}5$, ce valoare obținem (aproximativ)?

A 0,25

B 0,35

C 0,69

D 1,39

Verificare

? O aplicație web trebuie să genereze **token-uri de resetare a parolei**. Ce folosim?

A random.random() cu seed fix

B numpy.random.default_rng()

C modulul secrets

D un LCG propriu

Rezumatul orei 2

- ✓ Calculatorul produce numere **pseudo-aleatoare**: deterministe, dar care trec testele statistice
- ✓ LCG: $X_{n+1} = (aX_n + c) \bmod m$ – simplu, dar parametrii contează (RANDU!)
- ✓ Azi: **Mersenne Twister** (Python), **PCG64** (NumPy); **secrets** pentru securitate
- ✓ **Seed** fix → reproductibilitate; replicări cu **fluxuri independente**
- ✓ **Transformarea inversă** $X = F^{-1}(U)$: exponențiala $X = -\ln(U)/\lambda$; discrete prin intervale
- ✓ Acceptare – respingere, Box–Muller; în practică: **rng.exponential, rng.choice…**

Ora 3

Metoda Monte Carlo

1. Ideea și istoria metodei
2. Estimarea lui π: eroare, convergență, interval de încredere
3. Integrare Monte Carlo și reducerea variației
4. Analiza riscului unui proiect; aplicații moderne

Ce este metoda Monte Carlo?

Metoda Monte Carlo

Rezolvarea numerică a unei probleme prin **eșantionare aleatoare repetată**: generăm multe scenarii aleatoare, calculăm rezultatul pentru fiecare și îl analizăm statistic.

după Holban, cap. 9

- Sensul 1: calculul unor mărimi **deterministe** (integrale, arii, π) cu ajutorul aleatorului
- Sensul 2: înlocuirea unui fenomen real cu un **experiment statistic** (riscul unui proiect, vânzări)
- Model static, stocastic – exact tipul din clasificarea de săptămâna trecută

O istorie scurtă a metodei

1777

Buffon: probabilitatea ca un ac aruncat să taie o linie → estimarea lui π

1946

Ulam se întreabă cât de des iese un pasians – și propune să-l joace de multe ori pe calculator

1947–48

von Neumann, Metropolis: difuzia neutronilor, simulată pe ENIAC

1949

Articolul **„The Monte Carlo Method”** (Metropolis & Ulam); numele vine de la cazinoul din Monaco

1953

Algoritmul Metropolis: baza metodelor MCMC (săptămâna 3)

azi

grafică 3D, finanțe, fizică, AI, fiabilitate, planificare

Acul lui Buffon (1777)

$$P(\text{acul taie o linie}) = 2l / (\pi\, d)$$

l	lungimea acului ($l \le d$)
d	distanța dintre liniile paralele de pe podea
$l = d$	$P = 2/\pi \approx 0{,}637$ → aruncând acul de N ori: $\pi \approx 2N$ / (nr. de intersecții)

Primul „algoritm Monte Carlo”: o constantă matematică estimată dintr-un experiment aleator repetat. Același principiu îl vom folosi pe calculator.

Rețeta Monte Carlo

Cei cinci pași ai simulării Monte Carlo (Holban, cap. 9.2):

1 — Model

$y = f(x_1, \ldots, x_q)$: ce calculăm dintr-un scenariu

2 — Intrări aleatoare

generăm un scenariu din distribuțiile intrărilor

3 — Evaluare

calculăm y_i și îl memorăm

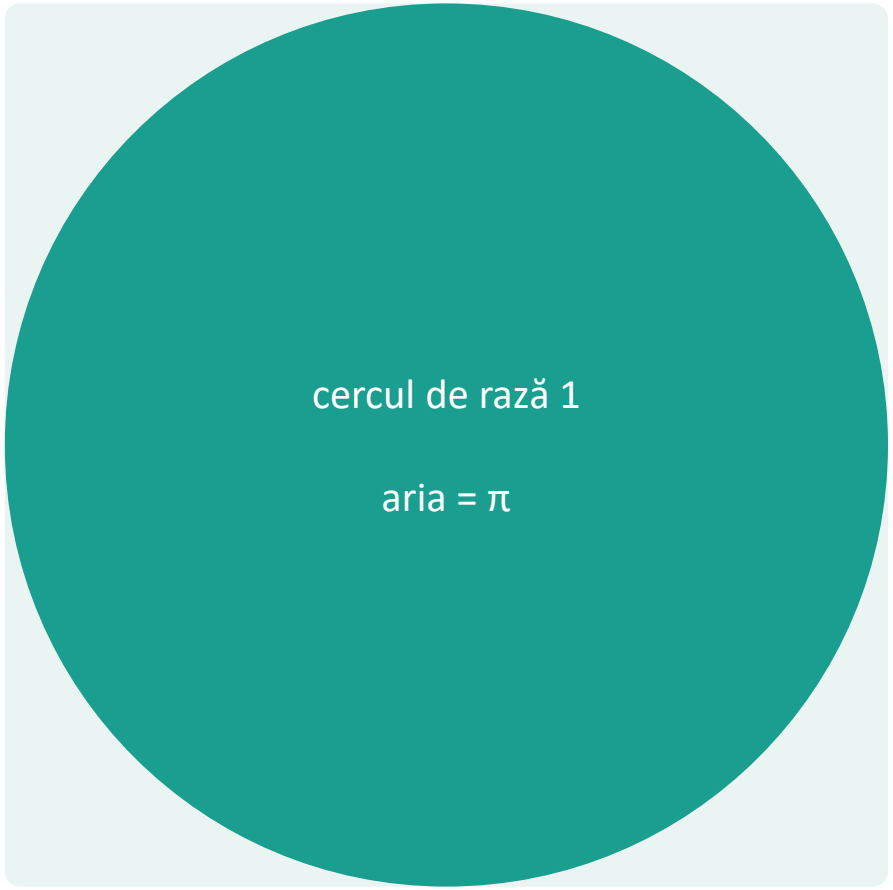
4 — Repetare

pentru i = 1 … N (N mare)

5 — Analiză

medie, histogramă, percentile, **interval de încredere**

Exemplul clasic: estimarea lui π



Pătratul $[-1, 1] \times [-1, 1]$ are aria **4**

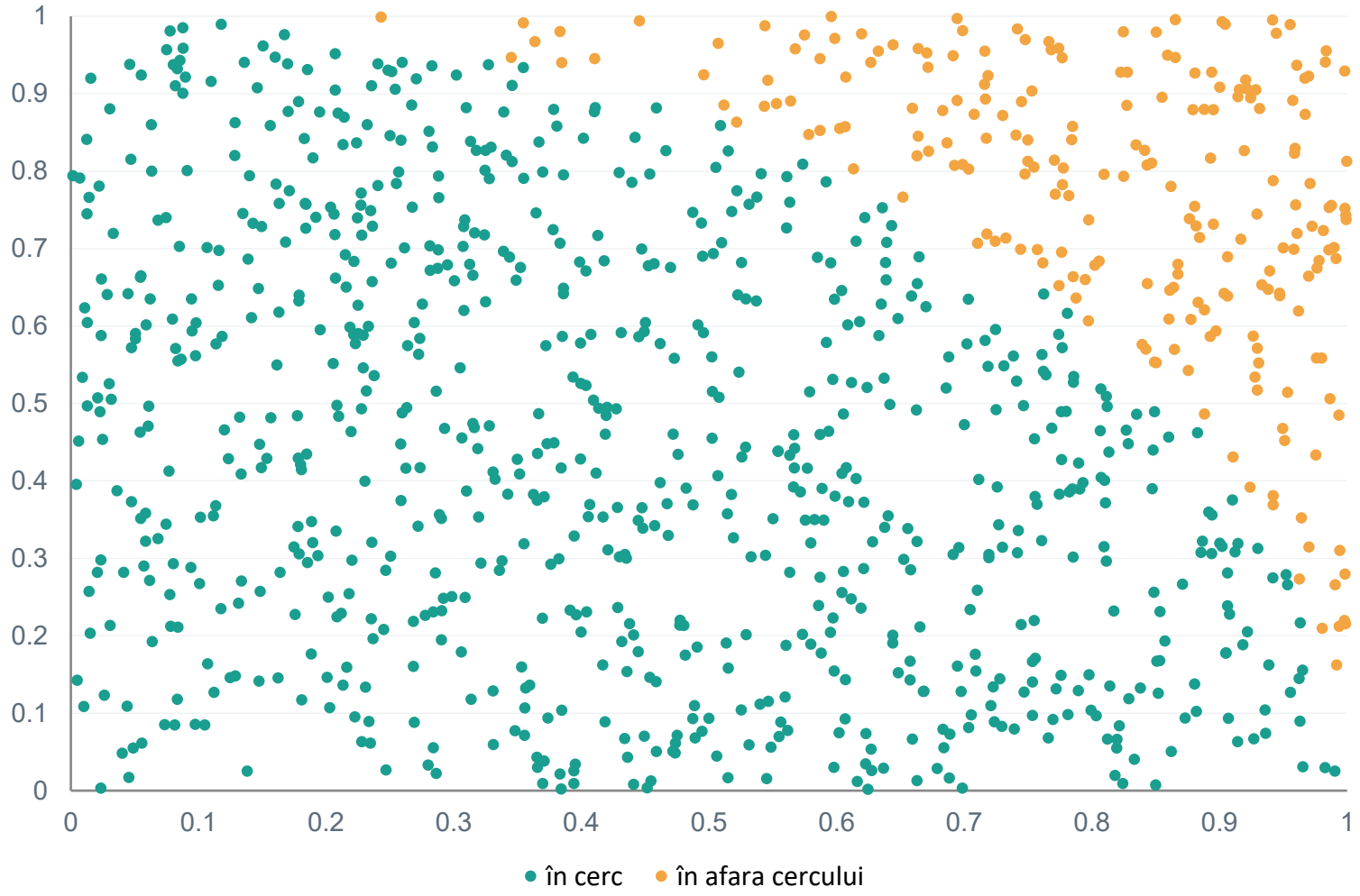
Cercul înscris $x^2 + y^2 \le 1$ are aria **π** → raportul π/4

Aruncăm N puncte uniform în pătrat; k cad în cerc

$\pi \approx 4 \cdot k / N$

Ca în Holban, cap. 9; în graficul următor folosim doar sfertul din dreapta-sus – raportul este același

1 000 de puncte aruncate: π ≈ 3,12



- **780** din 1 000 de puncte au căzut în sfertul de cerc
- π ≈ 4 · 780 / 1 000 = **3,12**
- Eroare: ≈ 0,02 – corect doar la prima zecimală
- Întrebare: câte puncte trebuie pentru 3 zecimale?

Estimarea lui π în NumPy

```
import numpy as np

def estimeaza_pi(N, rng):
    x = rng.random(N)
    y = rng.random(N)
    k = np.count_nonzero(x**2 + y**2 <= 1)
    return 4 * k / N

rng = np.random.default_rng(2026)
for N in [10**2, 10**4, 10**6]:
    print(N, estimeaza_pi(N, rng))
```

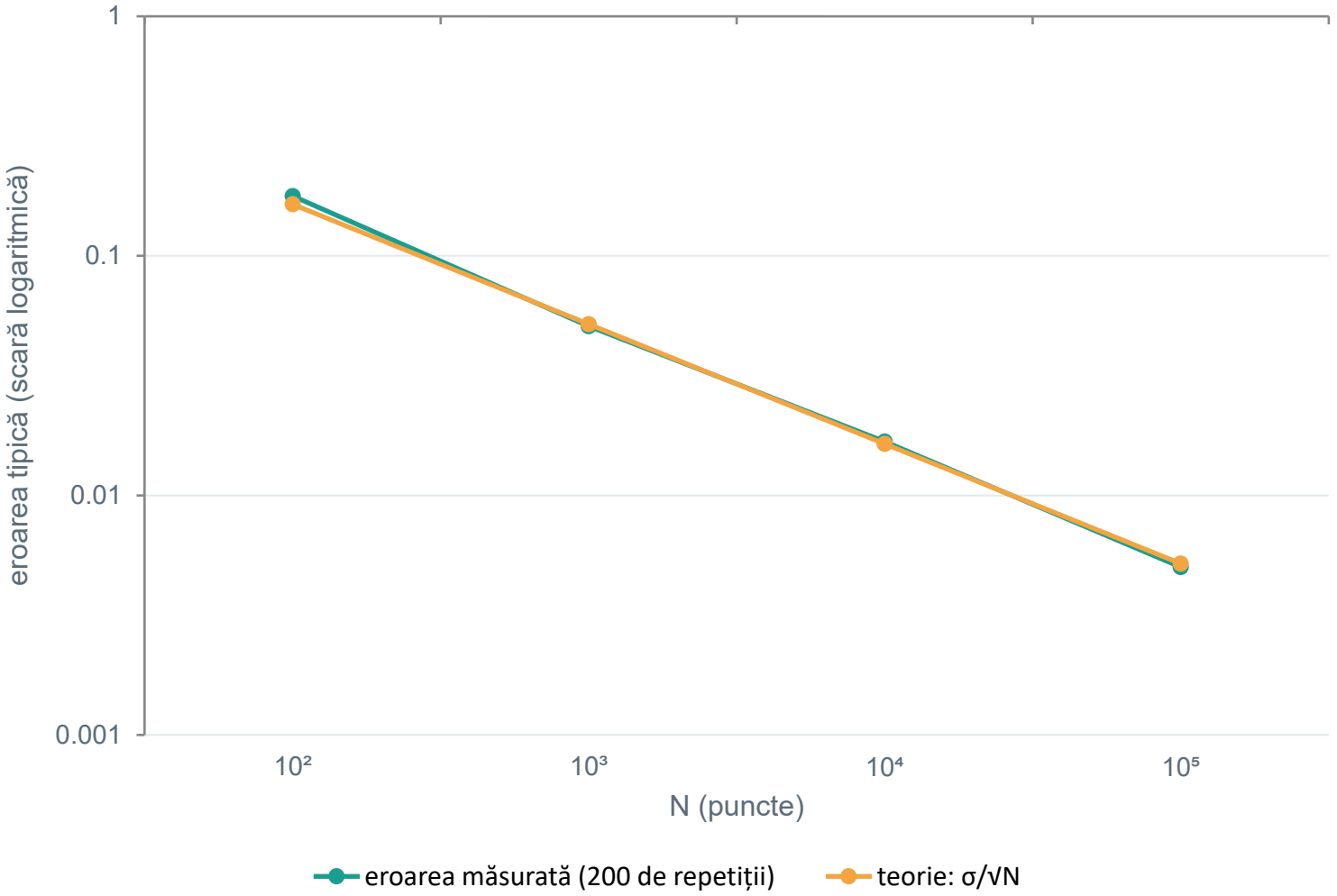
- Vectorizat: fără bucle Python
- 10^7 puncte în sub o secundă
- Pentru fiecare N folosim alte numere aleatoare
- Rezultatul este **o variabilă aleatoare** – depinde de numerele extrase

Estimări pentru N tot mai mare

N (puncte)	Estimarea lui π	Eroarea |estimare − π|	Eroarea relativă
10^2	2,68000	0,46159	14,693%
10^3	3,15600	0,01441	0,459%
10^4	3,14400	0,00241	0,077%
10^5	3,14396	0,00237	0,075%
10^6	3,13937	0,00222	0,071%
10^7	3,14115	0,00044	0,014%

O singură rulare pentru fiecare N; π = 3,14159…

Eroarea scade ca $1/\sqrt{N}$



- Fiecare ×10 la N → eroarea ÷ **√10 ≈ 3,16**
- Măsurătorile urmează exact curba teoretică
- N = 10^4: eroare tipică ≈ **0,017**
- N = 10^5: eroare tipică ≈ **0,005**

Legea erorii Monte Carlo

$$eroarea \approx \sigma / \sqrt{N}$$

σ	abaterea standard a unui singur rezultat (depinde de problemă)
N	numărul de repetări (puncte, scenarii, rulări)
$\times 100$	ca să câștigăm **o zecimală** în plus, avem nevoie de **100 de ori** mai multe repetări

Vestea proastă: convergența este **lentă**. Vestea bună: nu depinde de **dimensiunea** problemei și se paralelizează perfect.

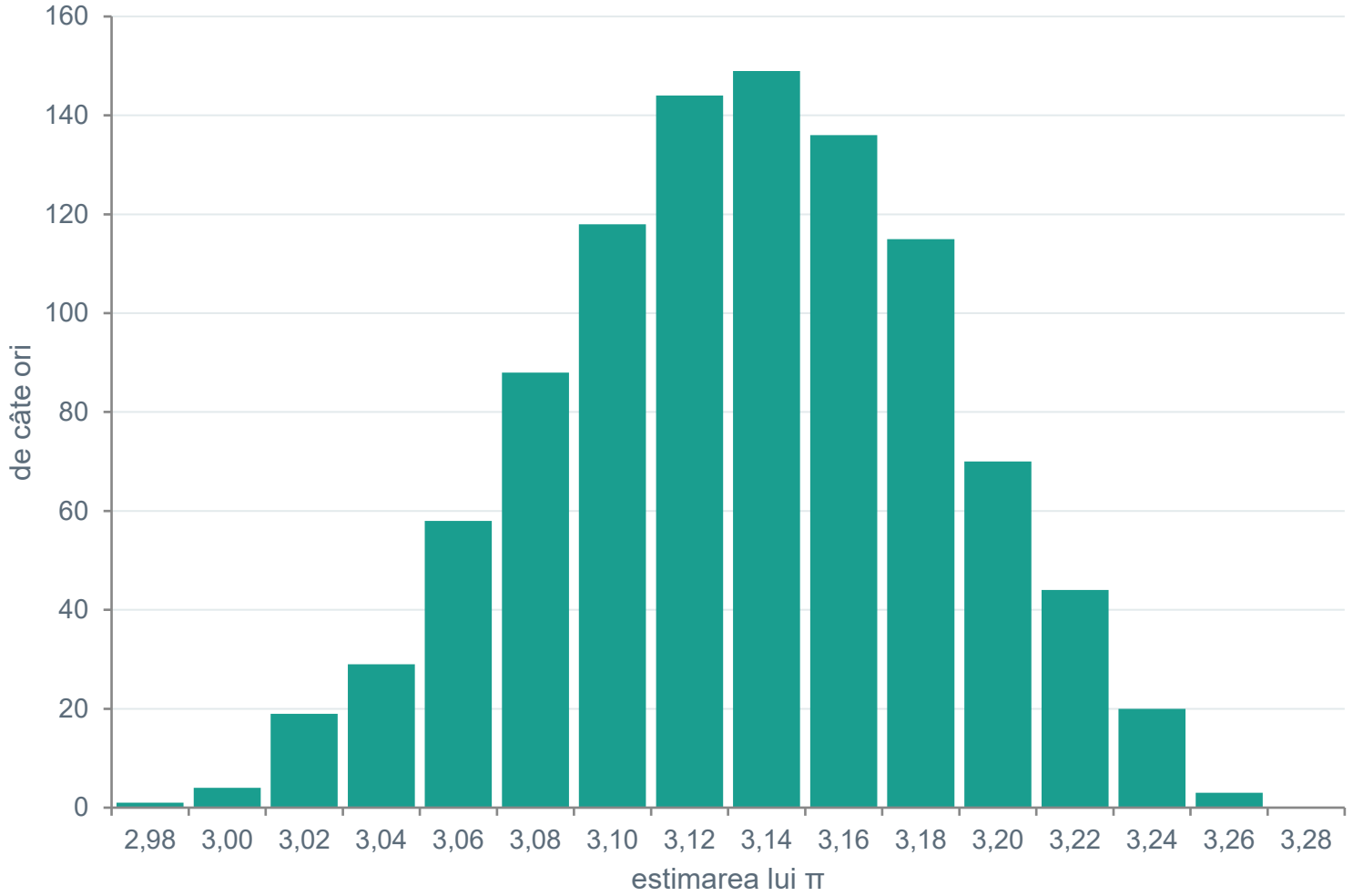
Legea numerelor mari și teorema limită centrală

Două teoreme pe care se sprijină totul

Legea numerelor mari: media a N repetări independente → valoarea adevărată. **Teorema limită centrală**: pentru N mare, media este aproximativ **normală**, cu abaterea standard σ/√N.

- Prima garantează că metoda **converge**
- A doua ne spune **cât de repede** și ne dă **intervalul de încredere**
- Merge oricare ar fi distribuția unui singur rezultat (dacă σ este finit)

TLC în acțiune: 1 000 de estimări ale lui π (N = 1 000)



- Forma de **clopot** a normalei
- Media estimărilor: **3,142**
- Abaterea standard: **0,0513** (teorie: 0,0519)
- Fiecare punct dă doar 0 sau 4 – media lor este totuși normală

Intervalul de încredere de 95%

$$\bar{x} \pm 1{,}96 \cdot s / \sqrt{N}$$

$\bar{x}$	media celor N rezultate
s	abaterea standard de selecție a rezultatelor
$1{,}96$	din normala standard: 95% din valori sunt între −1,96 și 1,96

s/√N se numește **eroarea standard** a mediei. Pentru N mic (sub ~30) se folosește distribuția t Student în loc de 1,96 (săptămâna 6).

Exemplu calculat: π cu N = 10 000

Mărime	Valoare
Media x̄ (estimarea)	3,1400
Abaterea standard s (a rezultatelor 0 / 4)	1,6434
Eroarea standard s/√N	0,0164
Semi-lățimea 1,96 · s/√N	0,0322
Intervalul de încredere 95%	[3,108; 3,172]
Conține valoarea adevărată π = 3,1416?	da

Cum interpretăm corect intervalul?

- **Corect**: dacă repetăm tot experimentul de multe ori, ~95% din intervalele construite astfel **conțin** valoarea adevărată
- **Greșit**: „valoarea adevărată are 95% șanse să fie în acest interval anume”
- Intervalul se **îngustează** ca 1/√N: de 4 ori mai multe rulări → interval de 2 ori mai îngust
- Un rezultat fără interval este o **anecdotă**, nu o concluzie

✓ **Formatul de raport**

$\pi \approx$ **3,140 ± 0,032** (IÎ 95%, N = 10 000, seed 10)

Câte repetări ne trebuie?

$$N \approx (1{,}96 \cdot \sigma / E)^2$$

E	marja de eroare dorită (semi-lățimea intervalului)
σ	estimată dintr-o **rulare pilot** mică
exemplu	π cu E = 0,01: σ ≈ 1,64 → N ≈ (1,96 · 1,64 / 0,01)² ≈ **103 599**

Procedura practică: rulare pilot (ex. N = 1 000) → estimăm σ → calculăm N → rulăm studiul principal.

Integrare Monte Carlo (metoda mediei)

$$\int_a^b f(x)\,dx \approx (b-a)\cdot (1/N)\,\Sigma\, f(x_i), \quad x_i \sim U(a,b)$$

ideea	integrala = (b − a) × valoarea **medie** a lui f pe interval
exemplu	∫₀¹ e^(−x²) dx = 0,746824… (nu are primitivă elementară)
eroarea	tot σ/√N, cu σ = abaterea standard a lui f(X)

Metoda „hit-or-miss” (ca la π) și metoda mediei („sample-mean”) sunt descrise în Holban, cap. 9.1.

Hit-or-miss vs metoda mediei

Hit-or-miss

- aruncăm puncte (x, y) sub un dreptunghi
- numărăm câte cad sub curbă
- rezultat ($N = 10^4$): **0,7423 ± 0,0086**
- abaterea unui rezultat: 0,44

vs

Metoda mediei

- generăm doar x și calculăm f(x)
- facem media valorilor
- rezultat ($N = 10^4$): **0,7450 ± 0,0039**
- abaterea unui rezultat: 0,20 → interval de ~2 ori mai îngust

De ce Monte Carlo pentru integrale?

10^{10}

puncte pe o grilă cu doar 10 valori pe axă, în **10 dimensiuni**

$\sigma/\sqrt{N}$

eroarea Monte Carlo **nu depinde** de numărul de dimensiuni

$d > 4–5$

de la câteva dimensiuni în sus, Monte Carlo devine de obicei alegerea practică

- În 1D, metodele clasice (trapeze, Simpson) sunt mult mai bune
- În dimensiuni mari (finanțe, fizică, randare): Monte Carlo câștigă

Reducerea varianței: aceeași precizie, mai puține rulări

1 Variabile antitetice

Folosim perechi (U, 1 − U): erorile se compensează.

2 Numere aleatoare comune

Comparăm două variante cu **aceleași** numere aleatoare (S6).

3 Variabile de control

Corectăm cu o mărime corelată a cărei medie o știm exact.

4 Eșantionare prin importanță

Eșantionăm mai des regiunile care contează (evenimente rare).

Exemplu: variabile antitetice pentru $\int_0^1 e^{(-x^2)}\,dx$

± 0,0039

metoda mediei, 10 000 de evaluări

± 0,0008

antitetic: 5 000 de perechi (U, 1 − U) = tot 10 000 de evaluări

×25

reducerea variației: echivalent cu de ~25 de ori mai multe rulări simple

- Funcția este monotonă, deci f(U) și f(1 − U) sunt **negativ corelate**
- Cost suplimentar: practic zero – o linie de cod

Monte Carlo ca experiment statistic

Riscul unui proiect software

1. Problema: cât durează proiectul?
2. Modelul: trei activități cu durate incerte
3. Simularea și interpretarea rezultatelor
4. Lecția: suma valorilor „cel mai probabil” nu este un plan

Proiect: un produs eLearning cu trei activități

Activitate	Optimist	Cel mai probabil	Pesimist
Scrierea conținutului	4 zile	6 zile	8 zile
Interfața grafică	5 zile	7 zile	9 zile
Integrarea elementelor multimedia	2 zile	4 zile	6 zile
Total (activități succesive)	11 zile	17 zile	23 zile

Estimările în trei puncte ale echipei (Holban, cap. 9, tabelul 9.8)

Modelul Monte Carlo al proiectului

```
import numpy as np
rng = np.random.default_rng(2024)
N = 10_000

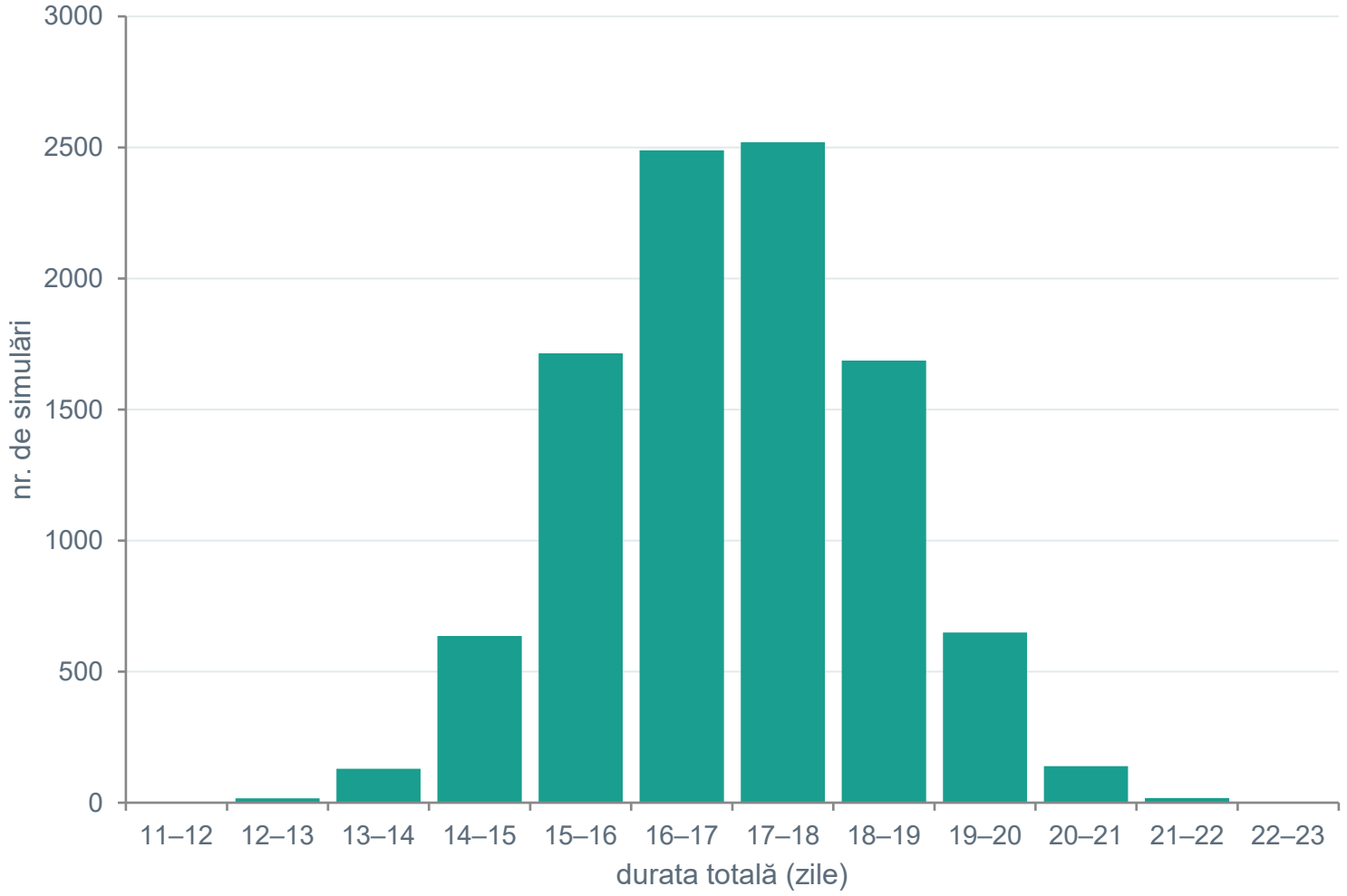
# (min, cel mai probabil, max) pe activitate
activitati = [(4, 6, 8), (5, 7, 9), (2, 4, 6)]

durata = sum(rng.triangular(a, m, b, N)
             for a, m, b in activitati)

print(durata.mean())                    # durata medie
print((durata <= 17).mean())      # P(durata ≤ 17)
print(np.percentile(durata, 90)) # termenul P90
```

- Fiecare activitate: distribuție **triunghiulară** (min, mod, max)
- 10 000 de proiecte simulate într-un singur apel
- Holban folosește uniforma; triunghiulara folosește și valoarea cea mai probabilă
- Rezultatul: o **distribuție** a duratei, nu un număr

Distribuția duratei proiectului (10 000 de simulări)



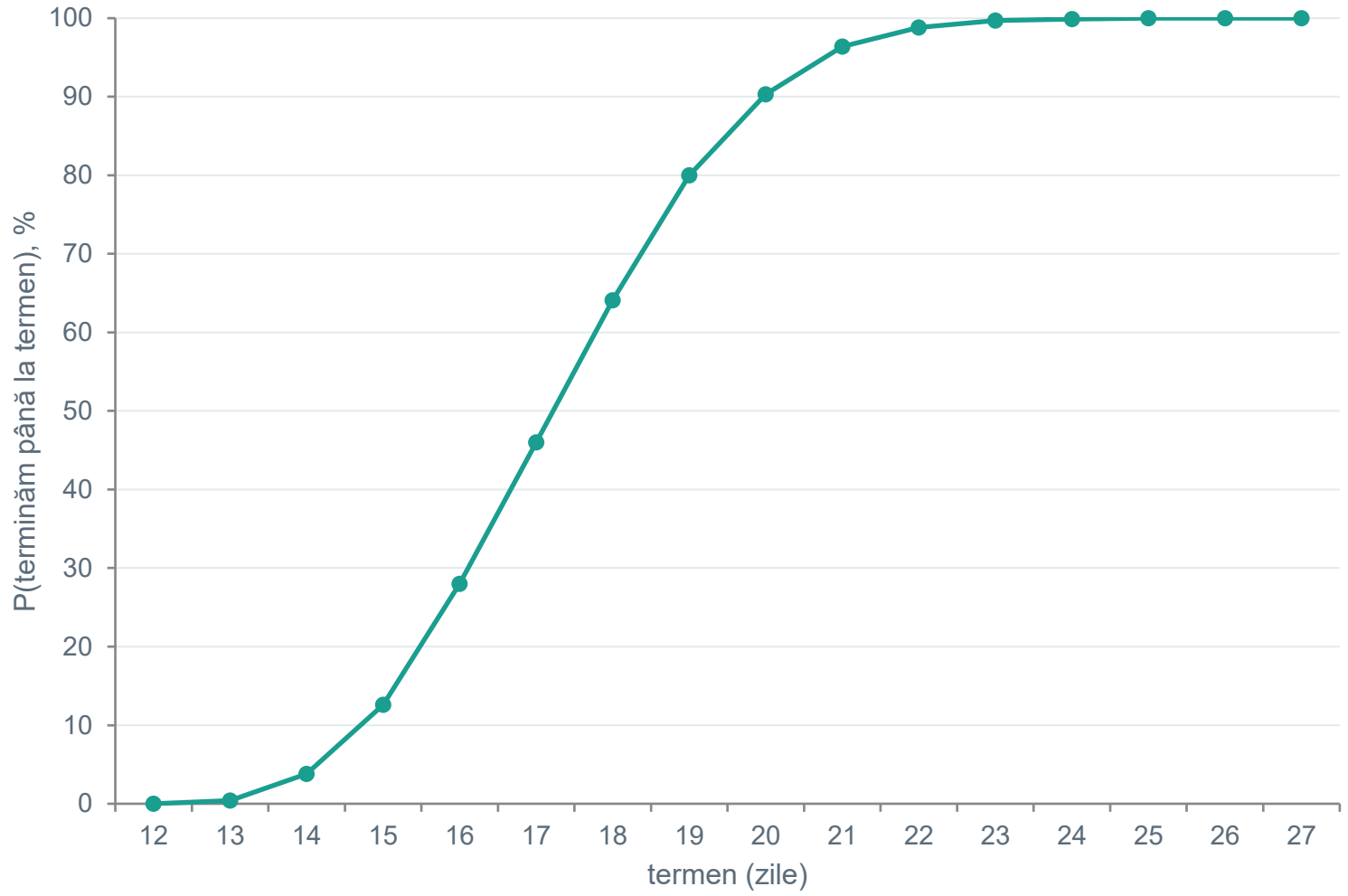
- Durata medie: **17,0 zile**
- P(durata ≤ 17 zile) ≈ **50%**
- Termen cu 80% siguranță (P80): **18,2 zile**
- Termen cu 90% siguranță (P90): **18,8 zile**

Varianta realistă: întârzierile sunt mai probabile decât avansurile

Activitate	Optimist	Cel mai probabil	Pesimist
Scrierea conținutului	4 zile	5 zile	9 zile
Interfața grafică	5 zile	6 zile	11 zile
Integrarea elementelor multimedia	2 zile	3 zile	7 zile
Plan naiv (suma valorilor „cel mai probabil”)		14 zile	

Estimări asimetrice: fiecare activitate poate întârzia mult, dar poate câștiga puțin

Probabilitatea cumulată de terminare



- Plan naiv de 14 zile: doar **3,8%** șanse
- Durata medie: **17,3 zile**
- P80: **19 zile**; P90: **20 zile**
- Suma valorilor „cel mai probabil” ≠ durata cea mai probabilă

Analiza de risc Monte Carlo: pașii

- Identificăm **variabilele de risc** (duratele, costurile)
- Determinăm **plaja de valori** și distribuția fiecăreia (min / mod / max)
- Identificăm **legăturile** dintre variabile (dependențe, corelații)
- Simulăm de multe ori și **analizăm statistic**: medie, P50, P80, P90
- Comunicăm **probabilități**, nu un singur număr

H **Holban, cap. 9**

Aceeași metodă se aplică la costuri, bugete, stocuri, capacitate de servere sau timp de livrare.

Monte Carlo în informatică azi

1 Grafică: path tracing

Fiecare pixel = o integrală estimată din raze aleatoare (filme, jocuri).

2 Jocuri și AI: MCTS

Monte Carlo Tree Search: evaluarea mutărilor prin partide simulate (AlphaGo).

3 Algoritmi randomizați

Testul de primalitate Miller–Rabin, quicksort cu pivot aleator.

4 Finanțe

Riscul unui portofoliu, prețul opțiunilor – mii de scenarii de piață.

5 Fiabilitate

Probabilitatea ca un sistem distribuit să cadă, cu componente care se defectează.

6 Statistică bayesiană

MCMC: eșantionare din distribuții complicate (săptămâna 3).

Path tracing: legea $1/\sqrt{N}$ o vedeți pe ecran

pixel

culoarea = media luminii adusă de multe drumuri aleatoare ale razelor

4× raze

pentru un zgomot de 2 ori mai mic – exact legea σ/√N

denoising

rețele neurale care „curăță” zgomotul imaginilor cu puține raze pe pixel

- Imaginile randate cu puține raze pe pixel arată „granulate”: acesta este zgomotul Monte Carlo
- AI + Monte Carlo: puține eșantioane, apoi reducerea zgomotului cu o rețea neurală

Greșeli frecvente în Monte Carlo

- Prea **puține** repetări, fără să calculăm marja de eroare
- Raportarea unui număr **fără interval de încredere**
- Același seed în toate replicările → rezultate identice, nu independente
- Distribuții de intrare **alese pentru comoditate** (uniformă peste tot)
- Ignorarea **dependențelor** dintre intrări
- Confundarea mediei cu un termen sigur (planul de 17 zile!)

✓ **Verificare rapidă**

Dublați N: dacă rezultatul se schimbă mai mult decât marja raportată, ceva nu e în regulă.

Verificare

? O estimare Monte Carlo are eroarea ≈ 0,02 cu N = 10 000. Aproximativ ce eroare vom avea cu **N = 1 000 000**?

A 0,0002

B 0,002

C 0,01

D 0,2

Verificare

? Din N = 100 de rulări am obținut media $\bar{x}$ = 20 minute și abaterea standard s = 5 minute. Care este intervalul de încredere de **95%**?

A 20 ± 5

B 20 ± 9,8

C 20 ± 0,98

D 20 ± 0,5

Verificare

? Trei activități succesive au duratele cele mai probabile 5, 6 și 3 zile, dar pot întârzia mult mai mult decât pot câștiga. Planul de **14 zile** este:

A cel mai probabil rezultat

B un termen sigur

C probabil prea optimist

D imposibil de evaluat

Verificare

? O rulare pilot arată că abaterea standard a unui rezultat este σ = 2 minute. Câte repetări trebuie pentru o marjă de eroare de **± 0,1 minute** (95%)?

A 40

B 400

C aproximativ 1 540

D aproximativ 15 400

Rezumatul săptămânii 2

- ✓ Distribuțiile de lucru: uniformă, Bernoulli/binomială, **Poisson** (numărări), **exponențială** (timpi), normală, empirică
- ✓ Media și dispersia; **variabilitatea** contează, nu doar media
- ✓ Numere **pseudo-aleatoare**: PCG64 / Mersenne Twister; **seed** → reproductibilitate; **secrets** pentru securitate
- ✓ **Transformarea inversă** $X = F^{-1}(U)$ (ex. $X = -\ln(U)/\lambda$) și acceptare – respingere
- ✓ Monte Carlo: eroarea ≈ **$\sigma/\sqrt{N}$**; raportăm **$\bar{x} \pm 1{,}96 \cdot s/\sqrt{N}$**
- ✓ $N \approx (1{,}96\sigma/E)^2$; reducerea variației (antitetice)
- ✓ Riscul unui proiect: comunicăm **probabilități** (P80, P90), nu un singur număr

Săptămâna viitoare

Plimbări aleatoare și lanțuri Markov

- Un pieton care se mișcă la întâmplare: difuzia
- Stări, tranziții, matrice de probabilități
- PageRank, generare de text, algoritmul Metropolis

Cum a ajuns Google să ordoneze paginile web cu ajutorul unui „navigator aleator”?

Bibliografie pentru săptămâna 2

- A. M. Law – Simulation Modeling and Analysis, ed. 6, McGraw Hill, 2024 – cap. 6 (distribuții), 7 (generatoare), 8 (generarea variabilelor)
- S. M. Ross – Simulation, Academic Press – capitolele despre generarea variabilelor aleatoare
- Ș. Holban – Tehnici de modelare și simulare, cap. 4 (statistică) și cap. 9 (Monte Carlo)
- R. J. Gibbens – Computer Systems Modelling, Cambridge – generarea numerelor și variabilelor aleatoare
- N. Metropolis, S. Ulam – The Monte Carlo Method, Journal of the American Statistical Association, 1949
- M. E. O'Neill – PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation, 2014
- Documentația NumPy – Random Generator (numpy.org/doc/stable/reference/random)
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5